Dynamic Programming

A Deep-Dive for Competitive Programming

Theory – Patterns – Recognition – Code – Problems

Competitive Programming Notes

October 20, 2026

Preface

This document is a self-contained, deep-dive treatment of **Dynamic Programming (DP)** as it is actually used in competitive programming (Codeforces, AtCoder, CSES, ICPC, IOI, and similar venues). It is not a shallow cheat-sheet: every standard pattern is accompanied by the underlying theory, the recurrence derivation, a fully working `C++17` implementation, a discussion of time and memory complexity, and a curated problem set with worked editorial-style solutions.

The document is organised around two complementary threads:

- **Pattern recognition.** A dedicated chapter teaches you how to look at a fresh contest problem and *recognise* that it is a DP problem — what constraint sizes hint at, what keywords ("count the number of ways", "is it possible to", "minimum cost to", "expected value of") hint at, and what graph shape (chain, tree, interval, subset, DAG, automaton) hints at.
- **Pattern catalogue.** Each subsequent chapter covers one family of DP — the knapsack family, LIS/LCS, interval DP, tree DP, DAG / path-counting, bitmask DP, digit DP, game DP, matrix exponentiation, string / automaton DP, probability DP, and finally advanced optimisations (divide-and-conquer, Knuth, Convex Hull Trick, Li Chao tree, alien's trick).

How to read this book. Each chapter is largely self-contained, but the recognition chapter assumes familiarity with the introductory chapter, and the optimisation chapter assumes you understand the patterns it optimises. Code is in portable C++17. Every snippet is meant to be lifted verbatim into a contest submission — standard `bits/stdc++.h` header, fast I/O, `long long` alias `ll` for 64-bit safety, 0-indexed arrays unless stated otherwise.

Compilation. This PDF is meant to be compiled with `pdflatex` on a stock TeX Live installation. No `--shell-escape` is required; we use `listings` rather than `minted` for portability. Run:

```
pdflatex main.tex   &&  pdflatex main.tex
```

twice so that the table of contents and cross-references settle.

Table of Contents

Preface **1**

1 Introduction to Dynamic Programming **6**
1.1 What is dynamic programming? . . . 6
1.2 A canonical first example: Fibonacci . . . 6
1.3 Top-down vs bottom-up: a practical comparison . . . 8
1.4 The four ingredients of every DP solution . . . 8
1.5 Complexity of a DP . . . 9
1.6 When *not* to use DP . . . 9
1.7 Reading the rest of this book . . . 9

2 Pattern Recognition: How to Spot a DP Problem **11**
2.1 Why a recognition chapter? . . . 11
2.2 Reading constraint sizes . . . 11
2.3 Reading problem-shape keywords . . . 12
2.4 Reading the graph structure . . . 13
2.5 The four-ingredients test . . . 14
2.6 Recognition checklist (tear-out) . . . 15
2.7 Common false friends (problems that look like DP but are not) . . . 15
2.8 Recognition exercises . . . 16

3 The Knapsack Family **17**
3.1 Why the knapsack family comes first . . . 17
3.2 The 0/1 knapsack . . . 17
3.3 Unbounded knapsack . . . 19
3.4 Bounded knapsack . . . 20
3.5 Subset sum variants in detail . . . 20
3.6 Two-dimensional knapsack (cost & weight) . . . 21
3.7 Problem set . . . 21
3.8 Common pitfalls . . . 22

4 The LIS / LCS Family **23**
4.1 The unifying idea: DP on indices into a sequence . . . 23
4.2 Longest Increasing Subsequence . . . 23
4.3 Longest Common Subsequence . . . 25
4.4 Edit distance in detail . . . 26
4.5 Problem set . . . 26
4.6 Common pitfalls . . . 27

5 Interval DP **28**
5.1 The unifying idea . . . 28
5.2 General interval DP template . . . 28
5.3 Matrix Chain Multiplication . . . 29
5.4 Burst Balloons . . . 30
5.5 Merging stones (Yandex / Leetcode 1000) . . . 30
5.6 Optimal Binary Search Tree . . . 31
5.7 Bracket sequences . . . 32
5.8 Problem set . . . 32
5.9 Common pitfalls . . . 33

6 Tree DP **34**
6.1 The unifying idea . . . 34
6.2 Setting up the recursion . . . 34
6.3 Tree diameter . . . 35
6.4 Maximum independent set on a tree . . . 35
6.5 Maximum matching on a tree . . . 36
6.6 Rerooting technique . . . 36
6.7 Tree knapsack . . . 37
6.8 Centroid decomposition (sketch) . . . 38
6.9 Problem set . . . 38
6.10 Common pitfalls . . . 39

7 DAG and Path-Counting DP **40**
7.1 The unifying idea . . . 40
7.2 Topological-sort-driven DP . . . 40
7.3 Grid path counting . . . 41
7.4 Longest path in a DAG . . . 41
7.5 DP on a DAG with edge weights . . . 42
7.6 Sum over paths . . . 42
7.7 Sum over subsets (SOS DP) . . . 43
7.8 Problem set . . . 43
7.9 Common pitfalls . . . 44

8 Bitmask DP **45**
8.1 When bitmask DP is the right tool . . . 45
8.2 The Travelling Salesman Problem (TSP) . . . 45
8.3 Hamiltonian path / cycle . . . 46
8.4 Assignment problem . . . 47
8.5 Minimum-weight vertex cover / Steiner tree . . . 47
8.6 Partition-into-pairs . . . 48
8.7 Iterating over submasks efficiently . . . 48
8.8 Problem set . . . 48
8.9 Common pitfalls . . . 49

9 Digit DP **50**
9.1 The unifying idea . . . 50
9.2 Canonical problem . . . 50
9.3 State and transition . . . 50
9.4 A first worked example: sum of digits equals S . . . 51
9.5 Variants and the parameter zoo . . . 52
9.6 Iterative (non-recursive) digit DP . . . 52
9.7 Two-bound digit DP . . . 52

9.8 Automaton-based digit DP . . . 53
9.9 Problem set . . . 54
9.10 Common pitfalls . . . 54

10 Game DP and Sprague–Grundy **55**
10.1 The unifying idea . . . 55
10.2 Definitions . . . 55
10.3 Canonical example: Nim . . . 56
10.4 Computing Grundy values . . . 56
10.5 Patterns of Grundy values . . . 56
10.6 Composite games and the XOR trick . . . 57
10.7 Misère play . . . 57
10.8 Partisan games . . . 57
10.9 Problem set . . . 58
10.10 Common pitfalls . . . 58

11 Matrix Exponentiation **59**
11.1 The unifying idea . . . 59
11.2 Linear recurrences and matrix form . . . 59
11.3 Fast matrix exponentiation . . . 60
11.4 Canonical example: Fibonacci to $F_{10^{18}}$ . . . 60
11.5 Tiling DP with matrix exponentiation . . . 61
11.6 Recurrence on a graph (number of walks) . . . 61
11.7 Linear recurrences with non-constant terms . . . 61
11.8 Cayley–Hamilton and Berlekamp–Massey . . . 62
11.9 Problem set . . . 62
11.10 Common pitfalls . . . 63

12 String and Automaton DP **64**
12.1 The unifying idea . . . 64
12.2 A first example: KMP automaton DP . . . 64
12.3 Aho-Corasick automaton DP . . . 65
12.4 Suffix automaton DP . . . 65
12.5 Longest Common Substring of two strings . . . 66
12.6 DP on suffix automaton: number of distinct substrings of length k . . . 66
12.7 Pattern match DP: edit distance with allowed operations . . . 67
12.8 Problem set . . . 67
12.9 Common pitfalls . . . 68

13 Probability and Expected Value DP **69**
13.1 The unifying idea . . . 69
13.2 Modular arithmetic prerequisites . . . 69
13.3 Linearity of expectation . . . 69
13.4 Canonical example: expected number of coin flips . . . 70
13.5 Random walks on graphs . . . 70
13.6 Expected value of “maximum element” . . . 71
13.7 Probability DP for “count of ways to reach a state” . . . 71
13.8 Variance, second moments . . . 71
13.9 Problem set . . . 71
13.10 Common pitfalls . . . 72

14 DP Optimisations **73**
14.1 Why a dedicated optimisation chapter . . . 73
14.2 Prefix-sum optimisation . . . 73
14.3 Monotonic queue / sliding-window optimisation . . . 74
14.4 Divide-and-conquer optimisation . . . 75
14.5 Knuth optimisation . . . 75
14.6 Convex Hull Trick . . . 76
14.7 Li Chao segment tree . . . 77
14.8 Alien's trick (Lagrangian relaxation) . . . 78
14.9 Other optimisations (sketches) . . . 79
14.10 Problem set . . . 79
14.11 Common pitfalls . . . 80

15 Conclusion, Master Checklist, and Pitfalls **81**
15.1 What you have learned . . . 81
15.2 Master recognition checklist . . . 82
15.3 Common pitfalls (cross-cutting) . . . 82
15.4 A workflow for contest DP problems . . . 83
15.5 Where to go next . . . 83
15.6 References . . . 84
15.7 Final words . . . 84

Chapter 1

Introduction to Dynamic Programming

1.1 What is dynamic programming?

Dynamic Programming (DP) is an algorithm-design technique that solves a problem by decomposing it into a collection of overlapping subproblems, solving each subproblem *once*, and storing the result so that subsequent occurrences of the same subproblem are answered by a lookup rather than by recomputation. The phrase was coined by Richard Bellman in the 1950s; the word "programming" here does not refer to writing code but to *tabular optimisation* (a usage that survives in "linear programming" and "non-linear programming").

In competitive programming, DP is arguably the single most important algorithmic paradigm. A large fraction of contest problems labelled "medium" or above either *are* DP problems or contain a DP reduction inside. Mastering DP means internalising a small library of recurrences and learning to spot when a fresh problem isomorphic to one of them has been placed in front of you.

The two pillars on which every DP solution stands are the following properties of the problem.

Definition 1.1: Optimal substructure

A problem exhibits *optimal substructure* when an optimal solution to the whole problem can be built from optimal solutions to its subproblems. Equivalently, the value of the global optimum is a function of the values of the optima of smaller instances.

Definition 1.2: Overlapping subproblems

A problem has *overlapping subproblems* when the same sub-instance is reached by many distinct recursive branches of the naive recursion. Without overlap, simple recursion already gives an efficient solution and DP is unnecessary.

Tip 1.1: Quick litmus test

If a problem has *optimal substructure* **and** *overlapping subproblems*, then DP is applicable. If it has optimal substructure but no overlap, plain recursion (or divide and conquer) is enough. If it has overlap but no optimal substructure, look at *greedy*, *minimax*, or *game theory* instead.

1.2 A canonical first example: Fibonacci

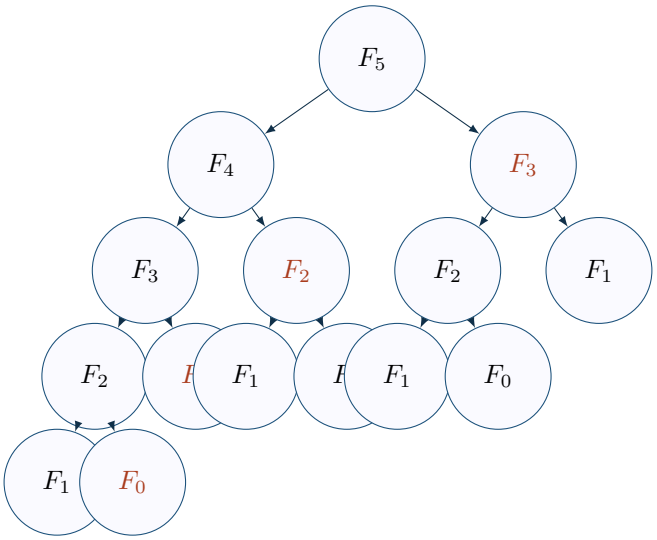
The Fibonacci sequence

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad (n \geq 2)$$

is the simplest non-trivial DP. A naive recursive implementation

```
int fib(int n){
    if(n<2) return n;
    return fib(n-1)+fib(n-2);
}
```

has exponential cost $\Theta(\varphi^n)$ because each call to `fib(n)` spawns two more calls that themselves spawn two more, and so on, until they hit the base case. The call tree, drawn below, contains the same call `fib(2)` three times, `fib(1)` five times, and `fib(0)` three times — this is precisely the "overlapping subproblems" property in action.



Nodes drawn in red are recomputed; this is the work that DP will eliminate.

1.2.1 Top-down with memoization

The standard fix is to remember the result of every `fib(k)` we have already computed. In C++17 we can use a hash map or a static array:

```
long long fib_memo(int n){
    static std::map<int,long long> memo;
    if(n < 2) return n;
    auto it = memo.find(n);
    if(it != memo.end()) return it->second;
    return memo[n] = fib_memo(n-1) + fib_memo(n-2);
}
```

This version still has the same logical structure as the naive recursion — it spends its first call descending to the base case, then on the way back fills in entries that subsequent branches simply look up. The result is $O(n)$ time and $O(n)$ recursion-depth memory. This style is called **top-down DP** or **memoization**.

1.2.2 Bottom-up with a table

Alternatively, we can simply compute the entries in the natural order:

```
long long fib_tab(int n){
    if(n < 2) return n;
    std::vector<long long> f(n+1);
    f[0]=0; f[1]=1;
    for(int i=2;i<=n;i++) f[i]=f[i-1]+f[i-2];
    return f[n];
}
```

Here we compute $F_2, F_3, \ldots, F_n$ in increasing order; when we need F_{i-1} and F_{i-2} they are already computed. This is **bottom-up DP** or **tabulation**. It is $O(n)$ time and $O(n)$ memory; with the observation that we only need the last two values, we can reduce to $O(1)$ memory.

1.3 Top-down vs bottom-up: a practical comparison

Aspect	Top-down (memoized)	Bottom-up (tabulated)
Control flow	Follows the natural recursion; only computes states actually needed.	Iterates over all states in a fixed order.
Memory	Recursive call stack + memo table; larger constant.	Flat array; smaller constant, often optimisable to $O(1)$.
Speed	Slight overhead per call (function call + map lookup).	Faster in practice; tight loops vectorise well.
Implementation risk	Stack overflow for deep recursion (e.g. $n > 10^6$).	None: pure iteration.
Easier to write?	Yes, when the recurrence is naturally recursive.	Yes, when the order is simple.

Tip 1.2: Rule of thumb

Use top-down during the contest while you are still debugging the recurrence — it is shorter and self-documents the recurrence. Once the recurrence is correct and the problem has tight limits, convert to bottom-up, especially if you can prune to $O(\sqrt{n})$ or $O(\log n)$ memory.

1.4 The four ingredients of every DP solution

Every DP solution you will ever write consists of four ingredients. Learn to identify them explicitly — it is the single most useful mental habit in contest DP.

(1) State. A tuple of indices, parameters, or counts that uniquely identifies a sub-instance. For Fibonacci the state is a single integer n. For 0/1 knapsack the state is (i, w) — "consider items $1..i$, knapsack capacity is w". For bitmask DP on n items, the state is a bitmask $S \subseteq \{1, \ldots, n\}$.

(2) Transition. A rule that expresses the value of a state in terms of values of strictly smaller states. For Fibonacci: $F_n = F_{n-1} + F_{n-2}$. For 0/1 knapsack: $dp[i][w] = \max(dp[i-1][w],\ dp[i-1][w-w_i] + v_i)$, where the two options are "skip item i" and "take item i".

(3) Base case. The smallest states whose values are known a priori. For Fibonacci: $F_0 = 0, F_1 = 1$. For knapsack: $dp[0][w] = 0$ for all w.

(4) Answer. The state (or combination of states) whose value is the solution to the original problem. For Fibonacci: F_n. For 0/1 knapsack: $\max_w dp[n][w]$.

Tip 1.3: The mental script

Whenever you read a problem you suspect is DP, write down the four ingredients *before* you write any code. If you cannot crisply state the four ingredients, you do not yet understand the recurrence — and coding now will only waste time. This discipline is more important than any single algorithm you will learn in the rest of the book.

1.5 Complexity of a DP

If a DP has S reachable states and each state's value is computed from T strictly smaller states, then the time complexity is $\Theta(S \cdot T)$ and the memory complexity is $\Theta(S)$ (or less, if states can be discarded). This is the *state-counting* way of estimating DP complexity:

Lemma 1.1: State-counting bound

If every transition reads at most T previously computed states and the state space has cardinality S, then the algorithm runs in $O(S \cdot T)$ time and $O(S)$ memory, assuming each state-value lookup is $O(1)$.

This bound is the one to use at the keyboard. For 0/1 knapsack with n items and capacity W, $S = n \cdot W$ and $T = 2$, giving $O(nW)$ time and $O(nW)$ memory — and we can reduce memory to $O(W)$ using a rolling array.

Warning 1.1: Beware pseudo-polynomiality

$O(nW)$ is *pseudo-polynomial*: W is the magnitude of an input integer, not the length of the input. If $W = 10^9$, the algorithm uses $\sim 10^9$ cells per row and will TLE/MLE on any contest. Always sanity check that $n \cdot W \leq 10^7$ before committing to knapsack-style DP.

1.6 When *not* to use DP

DP is powerful but slow; do not reach for it before exhausting cheaper options. The following list, while not exhaustive, captures the most common mis-uses:

- **Greedy works.** If a problem admits a greedy choice property and a matroid / exchange argument, the greedy is $O(n \log n)$ or $O(n)$ and will beat any DP. *Examples:* activity selection, fractional knapsack, Huffman codes, the diameter-of-a-tree via two BFS.
- **Divide and conquer suffices.** If the subproblems do not overlap, plain D&C gives a tight recurrence $T(n) = 2T(n/2) + O(n)$ and DP is overkill. *Example:* merge sort, closest pair.
- **There is a closed form.** The Fibonacci recurrence has Binet's formula $F_n = (\varphi^n - \psi^n)/\sqrt{5}$; for very large n, compute it via matrix exponentiation in $O(\log n)$ rather than $O(n)$ DP. See Chapter 11.
- **The problem is NP-hard.** Some problems admit DP only with exponential state spaces (e.g. general TSP is $O(2^n \cdot n^2)$, which is feasible only for $n \leq 20$). If n is larger and the problem is genuinely NP-hard, look for approximation, heuristics, or branch-and-bound instead.

1.7 Reading the rest of this book

Each subsequent chapter follows the same template. After the introductory and recognition chapters, every pattern chapter contains:

1. A **canonical problem** that exhibits the pattern in its purest form.
2. The **recurrence derivation**, written out as a piece of mathematics — the recurrence, the base cases, the answer.
3. A **complexity discussion** in the state-counting form introduced above.
4. A **working C++17 implementation**, intended to be lifted verbatim into a contest submission.

5. A **common-variants** subsection: what else is the same pattern in disguise.
6. A **problem set** of 5–8 curated problems from CSES, AtCoder, Codeforces, and SPOJ, each with an editorial-style walkthrough and a reference C++ solution.

The last chapter is a *master checklist* of all the patterns and their recognition signals, suitable for printing on a single sheet and keeping next to the keyboard during a contest.

Chapter 2

Pattern Recognition: How to Spot a DP Problem

2.1 Why a recognition chapter?

Most contest DP problems are not labelled "DP". The setter will not tell you it is a knapsack in disguise, or an interval DP, or a digit DP. The single most valuable skill in contest DP is the ability to read a fresh problem statement and *recognise* it as a DP problem — ideally within ninety seconds, before you have even started thinking about a recurrence. This chapter is a focused drill on that recognition skill.

The skill decomposes into four sub-skills, each of which we treat in turn:

1. Reading **constraint sizes** to narrow down the complexity class (Section 2.2).
2. Reading **problem-shape keywords** to narrow down the DP family (Section 2.3).
3. Reading the **graph structure** of the input — chain, tree, DAG, subset, interval — to map to a chapter (Section 2.4).
4. Sanity-checking against the **four ingredients** test from Chapter 1 (Section 2.5).

2.2 Reading constraint sizes

In modern contests the problem statement always gives explicit upper bounds on every variable. Those bounds — more than the prose — *tell you the intended complexity*, and the intended complexity *tells you the DP family.*

The convention on Codeforces / AtCoder / CSES is that the time limit allows roughly 10^8 elementary operations per second. So the intended complexity is whatever fits in $\sim 10^8$ for the largest input. The following table is the field guide.

Largest n	**Likely complexity**	**Likely DP family**	**Examples**
$n \leq 20$	$O(2^n \cdot n^2)$	Bitmask DP, Hamiltonian path, TSP,set cover, Steiner tree on subsets.	TSP, minimum Hamiltonian path,graph colouring, vertex cover.
$n \leq 22$	$O(2^n \cdot n)$	Bitmask DP, simpler transitions,DP over subsets, SOS DP (with $O(n \cdot 2^n)$).	Sum over subsets,Steiner tree DP, assignment DP.

Largest n	Likely complexity	Likely DP family	Examples
$n \le 100$	$O(n^3)$	Interval DP, $O(n^2)$ states $\times$ $O(n)$ transition.	Matrix chain multiplication, burst balloons,optimal BST, merging stones, bracket DP.
$n \le 500$–1000	$O(n^2)$	Linear DP, LIS-style, edit distance,classical DP on pairs of indices.	LIS, LCS, editdistance, longest palindromic substring, planarpolygon triangulation.
$n \le 5000$	$O(n^2)$	Quadratic DP without log factors.	Knapsackon n items and $W \le 5000$, longest palindromicsubsequence in $O(n^2)$.
$n \le 10^5$	$O(n)$ or $O(n \log n)$	Linear DP, segment tree DP,coordinate-compressed DP, divide-and-conquer optimised DP.	LIS via patience sort, DP on DAG, monotonic CHT.
$n \le 10^6$	$O(n)$	Linear scan DP, sliding window DP, digit DPwith 10^6-length number.	Counting strings, automaton DPon input string.
$W \le 10^5$ (sum constraint)	$O(n \cdot W)$	Knapsack family, subsetsum, partition.	0/1 knapsack, partitionequal subset sum, count of subsets withtarget sum.
String length $\le 10^6$	$O(n)$ or $O(n \log n)$	Suffix automaton DP,Z-function DP, hashing-based DP.	Count distinctsubstrings, suffix automaton DP,string match DP.
$\lvert L\rvert, \lvert R\rvert \le 10^{18}$	$O(\log \lvert R\rvert \cdot \lvert\Sigma\rvert \cdot \lvert\text{states}\rvert)$	Digit DP on a huge range.	Count of numbers in$[L, R]$ divisible by k, sum of digits equals S.
Tree with n nodes	$O(n)$, $O(n \log n)$, $O(n^2)$ if subtree DP withmerge.	Tree DP, rerooting DP, heavy-light DP.	Diameter, tree independent set, max path, treeknapsack.

Tip 2.1: The two-second rule

Multiply the largest n by the largest plausible state-count, then read off the complexity. If that product is $\le 10^8$, the DP fits in the time limit. If it is between 10^8 and 10^9, you will need optimisations (Chapter 14). If it is above 10^9, you almost certainly have the wrong family.

2.3 Reading problem-shape keywords

Beyond the constraint sizes, the prose of the problem statement carries keywords that map almost deterministically to a DP family. Memorise the following lexicon.

Keyword in statement	What it usually means	Likely DP family
"count the number of ways" / "how many"	Counting problem; the answer isoften big, mod10^9+7 requested.	Almost always DP (counting DP).
"minimum cost to" / "maximum value of"	Optimisation over a sequence ofchoices.	DP; check if greedy first.
"is it possible to" / "can you"	Decision problem; treat yes/no as 0/1and ask for max.	DP (subset-sum-style).
"expected value of" / "probability that"	Random process; answer is arational, mod10^9+7 requested.	Probability / expected value DP.
"in how many ways can X happen by time T"	Markov-like process on timesteps.	Linear DP, automaton DP.
"the number of integers in $[L, R]$ such that ..."	Range of size up to10^{18}.	Digit DP (Chapter 9).
"partition the array into contiguous subarrays ..."	Cuts of a line.	Interval DP / linear DP with cut transitions.
"best order to multiply / merge"	Reordering operations with associativestructure.	Interval DP (Chapter 5).
"visit every city exactly once, minimum cost"	Permutation problem with$n \leq 20$.	Bitmask DP / TSP (Chapter 8).
"on a tree, choose a subset of vertices ..."	Subtree structure.	TreeDP (Chapter 6).
"Alice and Bob take turns ..., who wins"	Impartial game.	Game DP /Sprague-Grundy (Chapter 10).
"tiling a $2 \times n$ grid with dominos/trominos"	Counting tilings.	Linear DP; if n is huge, matrix exponentiation (Chapter 11).
"count the number of paths from $(0, 0)$ to (n, m) ..."	Grid paths.	Linear DP on a 2D state.
"k-th lexicographically smallest valid string"	Counting DP to binarysearch the next digit.	Digit-style DP on a custom automaton.
"count distinct substrings"	Suffix structure.	Suffix automaton /suffix array DP.
"find longest subsequence that ..."	Selection from a sequence.	LIS/ LCS family (Chapter 4).

Warning 2.1: Watch for "greedy-looking" DP

Statements with the words "minimum" or "maximum" are dangerous because greedy *almost* works on many of them but fails on a small edge case. Always construct two or three small adversarial tests before committing to greedy. If greedy fails, you fall back to DP.

2.4 Reading the graph structure

The third recognition axis is the *shape* of the input. The shape of the input almost always dictates the shape of the state space.

2.4.1 The state-shape taxonomy

Linear / 1-D. The input is an array $a[1..n]$. State is one index. *Examples:* LIS, max subarray sum, jump game.

Two-index / pair. State is (i, j). *Examples:* LCS, edit distance, palindrome DP, longest common substring.

Interval. State is $[l, r]$ with $l \le r$. *Examples:* matrix chain multiplication, burst balloons, merging stones, optimal BST. Chapter 5.

Subset / bitmask. State is $S \subseteq \{1, \ldots, n\}$, encoded as an n-bit integer. *Examples:* TSP, assignment, set cover relaxation. Chapter 8.

Tree. State is a subtree rooted at u, optionally augmented with a "second parameter" for rerooting. Chapter 6.

DAG. State is a vertex in a DAG; transitions follow topological order. Chapter 7.

Digit. State is (position, tight flag, ...) on the decimal / binary expansion of L, R. Chapter 9.

Automaton. State is a node in an automaton (suffix automaton, Aho-Corasick, KMP automaton). Chapter 12.

Game. State is a game position; transitions are valid moves. Chapter 10.

Tip 2.2: Shape implies state

Once you have decided on the shape, the state is essentially decided for you. If the input is a 1-D array, the state is one index. If the input is a tree, the state is a node (possibly with a parent / parameter). If the input is a range $[L, R]$, the state is a digit position. The art is in deciding *the shape*, not the state.

2.4.2 A worked recognition drill

Consider the following problem statement (paraphrased from a Codeforces round):

> *You are given an array a of n integers ($n \le 5000$) and an integer S ($S \le 5000$). You may partition the array into any number of contiguous subarrays; each subarray contributes (sum of the subarray) $\times$ (length of the subarray) to the cost. Find the minimum total cost to make the sum of costs exactly S.*

Reading the constraints: $n \le 5000$, $S \le 5000$. Their product is $\le 2.5 \times 10^7$, which fits in time. This rules out $O(n^3)$ interval DP and strongly suggests an $O(n \cdot S)$ knapsack-style DP. Reading the keywords: "partition the array into contiguous subarrays" and "minimum total cost" $\to$ linear DP with cut transitions; the additional parameter S enters the state. Reading the shape: 1-D array + a sum parameter $\to$ state is (i, s): "minimum cost to process prefix $1..i$ and accumulate cost sum s". This is essentially the bounded knapsack DP from Section 3.4 disguised.

The recognition was: constraints ($O(n \cdot S)$) + keyword ("partition" + "minimum cost") + shape (1-D + scalar parameter) $\Longrightarrow$ knapsack family. We did not need to invent the recurrence from scratch; we mapped to a known family.

2.5 The four-ingredients test

Once you suspect a DP, run the four-ingredients test from Chapter 1 (state, transition, base case, answer). If you can fill in all four with crisp definitions, you have a DP. If any one is fuzzy, you do not yet have a DP.

2.5.1 Worked four-ingredients tests

(a) 0/1 Knapsack. Given n items with weight w_i and value v_i and capacity W:

- **State:** (i, c) = "consider items $1..i$ with capacity c".
- **Transition:** $dp[i][c] = \max\{dp[i-1][c],\ dp[i-1][c-w_i] + v_i\}$ (skip / take).
- **Base:** $dp[0][c] = 0$ for all $c \in [0, W]$.
- **Answer:** $dp[n][W]$.

(b) Longest Increasing Subsequence. Given array a of length n:

- **State:** $dp[i]$ = "length of the longest increasing subsequence ending at index i".
- **Transition:** $dp[i] = 1 + \max_{j<i,\, a_j<a_i} dp[j]$ (or 1 if no such j).
- **Base:** $dp[1] = 1$ (a single element is an increasing subsequence of length 1).
- **Answer:** $\max_i dp[i]$.

(c) Travelling Salesman (bitmask DP). Given complete graph with $n \leq 18$ vertices and edge weights c_{ij}:

- **State:** (S, v) = "currently at vertex v, having visited exactly the set $S \subseteq \{1, \ldots, n\}$".
- **Transition:** $dp[S][v] = \min_{u \in S \setminus \{v\}} dp[S \setminus \{v\}][u] + c_{uv}$.
- **Base:** $dp[\{0\}][0] = 0$ (start at vertex 0, having visited only 0).
- **Answer:** $\min_v dp[\text{all}][v] + c_{v,0}$ (return to start), or $\min_v dp[\text{all}][v]$ if a return trip is not required.

2.6 Recognition checklist (tear-out)

When you read a fresh problem, walk through the following six questions in order. If you can answer "yes" to the first four and "no" to the last two, you are looking at a DP problem.

DP recognition checklist (use this every time)

1. Does the largest constraint suggest an $O(n^k)$, $O(2^n \cdot n^2)$, $O(n \cdot W)$, $O(\log |R| \cdot k)$, or other DP-shaped complexity?
2. Does the prose contain a keyword from the lexicon in Section 2.3?
3. Is the input shape one of: 1-D array, pair of indices, interval, subset, tree, DAG, digit-string, automaton, or game position?
4. Can you crisply state the state, transition, base case, and answer?
5. (Trap.) Is the problem solved by a greedy argument with an exchange proof? **If yes, use greedy.**
6. (Trap.) Is the state space large ($> 10^9$) even after the obvious parameter pruning? **If yes, you have the wrong family.**

2.7 Common false friends (problems that look like DP but are not)

- **Shortest path in a graph with negative weights.** Looks like DP (relax edges repeatedly), but is Bellman-Ford, not DP in the strict sense. Use Dijkstra if non-negative.

- **Maximum flow.** Looks like DP (paths augment flow), but is its own paradigm (Ford-Fulkerson, Dinic, etc.).
- **Stable marriage / bipartite matching.** Looks like DP over pairs, but is its own combinatorial algorithm (Gale-Shapley, Hungarian).
- **Subset sum with $n \leq 40$ and no constraint on W.** Here $O(nW)$ is exponential and the intended algorithm is meet-in-the-middle $O(2^{n/2})$ — not DP.

2.8 Recognition exercises

Each of the following is a real or paraphrased contest problem. For each, identify (i) the constraint-based complexity hint, (ii) the keyword, (iii) the shape, and (iv) the chapter of this book that covers the pattern. Answers are at the end of the chapter.

1. Given an $n \times m$ grid ($n, m \leq 1000$), count paths from $(1, 1)$ to (n, m) moving only right or down.
2. Given $n \leq 15$ cities and a complete graph with c_{ij}, minimum cost of a tour that visits every city at least once.
3. Given an integer $X \leq 10^{18}$, count integers in $[1, X]$ whose sum of digits equals S.
4. Given a tree with $n \leq 10^5$ nodes, each with a value, choose a subset of vertices such that no two chosen are adjacent and the sum of values is maximum.
5. Given a string of length $n \leq 10^5$, count the number of distinct substrings.
6. Given an array of $n \leq 100$ integers, find the minimum cost to multiply them in some order, where cost of multiplying $A \times B$ is $|A| \cdot |B|$.

Answers to recognition exercises

1. (i) $O(nm)$, fits. (ii) "count paths". (iii) 2-D grid. → Linear DP on a 2-D state. (Section 7.3.)
2. (i) $n \leq 15$ suggests $O(2^n \cdot n^2)$. (ii) "visit every city at least once". (iii) subset / permutation. → Bitmask DP / TSP. Chapter 8.
3. (i) $|R| \leq 10^{18}$ suggests $O(\log R \cdot k)$. (ii) "count integers in $[1, X]$ with sum of digits S". (iii) digit string. → Digit DP. Chapter 9.
4. (i) $n \leq 10^5$ suggests $O(n)$. (ii) "choose subset of vertices ... no two adjacent ... maximum sum". (iii) tree. → Tree DP, maximum independent set. Chapter 6.
5. (i) $n \leq 10^5$ suggests $O(n)$. (ii) "count distinct substrings". (iii) string / suffix. → Suffix automaton DP. Chapter 12.
6. (i) $n \leq 100$ suggests $O(n^3)$. (ii) "best order to multiply / merge". (iii) interval. → Matrix chain multiplication / interval DP. Chapter 5.

If you got all six right in under five minutes each, you have the recognition skill — the rest of the book is filling in the recurrences.

Chapter 3

The Knapsack Family

3.1 Why the knapsack family comes first

The knapsack family is the most universally applicable DP family in competitive programming. Once you internalise its recurrence, you will find that a remarkable fraction of contest problems labelled "medium" are simply knapsack in disguise: *subset sum*, *partition equal subset sum*, *target sum*, *count of subsets*, *bounded knapsack*, *unbounded knapsack*, and many "partition the array into contiguous subarrays" problems reduce to one or another variant.

The unifying idea is: *you have n objects, each with a non-negative integer weight (and optionally a value), and a knapsack of capacity W. You decide for each object whether to include it, and you ask: what is the maximum total value / number of ways / possibility?* Every problem in the family is a permutation of those four ingredients.

3.2 The 0/1 knapsack

3.2.1 Canonical problem

You are given n items. Item i has weight w_i and value v_i. You have a knapsack of capacity W. Each item can be used *at most once*. Find the maximum total value you can carry without exceeding W.

3.2.2 Recurrence derivation

State: $dp[i][c]$ = "maximum value achievable considering items $1, 2, \ldots, i$, with knapsack capacity c". The decision at item i is binary: either we skip it (the knapsack keeps capacity c and we inherit the answer from items $1..i-1$), or we take it (we pay w_i units of capacity and gain v_i units of value, and then inherit the best answer for items $1..i-1$ with the reduced capacity $c-w_i$). Hence:

$$\boxed{dp[i][c] = \max\left(\, dp[i-1][c],\;\; dp[i-1][c-w_i] + v_i \,\right)} \tag{3.1}$$

where the second option is available only if $c \geq w_i$, and otherwise we just take $dp[i-1][c]$.

Base case: $dp[0][c] = 0$ for all $c \in [0, W]$ — with no items considered, the best value is 0 regardless of capacity.

Answer: $dp[n][W]$.

3.2.3 Complexity

State count $S = n \cdot (W+1)$, transition count $T = 2$, so $O(n \cdot W)$ time and $O(n \cdot W)$ memory. We can reduce memory to $O(W)$ via the *rolling array* technique (Section 3.2.5).

3.2.4 C++17 implementation

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; ll W;
    cin >> n >> W;
    vector<ll> w(n), v(n);
    for(int i=0;i<n;i++) cin >> w[i] >> v[i];
    // dp[c] = max value achievable with capacity c, considering items so far.
    vector<ll> dp(W+1, 0);
    for(int i=0;i<n;i++){
        // Iterate c from high to low so that we read dp[c - w[i]] from
        // the PREVIOUS row (i.e. before item i was considered).
        for(ll c = W; c >= w[i]; c--){
            dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
        }
        // c < w[i] : item i does not fit, dp[c] is unchanged.
    }
    cout << dp[W] << "\n";
    return 0;
}
```

Tip 3.1: Why the high-to-low loop?

If we iterated c from low to high, by the time we reached c we would have already overwritten $dp[c - w_i]$ with the new value that *uses* item i — effectively giving item i the chance to be picked twice. That is exactly the *unbounded* knapsack. The direction of the loop is the *only* syntactic difference between 0/1 and unbounded knapsack.

3.2.5 Rolling array reduction

Observe that recurrence (3.1) only ever reads row $i - 1$. We therefore do not need the full $n \times (W + 1)$ matrix; a single row of length $W + 1$ suffices, provided we iterate c from high to low so that we never overwrite a value we still need to read. The code above uses exactly this trick.

3.2.6 Reconstruction of the chosen subset

For problems that ask not just for the maximum value but also for *which* items achieve it, store the decision bit:

```cpp
vector<vector<char>> take(n, vector<char>(W+1, 0));
for(int i=0;i<n;i++)
    for(ll c=W;c>=w[i];c--)
        if(dp[i-1][c-w[i]] + v[i] > dp[i-1][c]){
            dp[i][c] = dp[i-1][c-w[i]] + v[i];
            take[i][c] = 1;
        }
// Reconstruction: walk backwards from (n, W).
ll c = W;
vector<int> chosen;
for(int i=n-1;i>=0;i--){
    if(take[i][c]){ chosen.push_back(i); c -= w[i]; }
}
```

3.2.7 Common variants

Subset sum (decision). Set $v_i = w_i$ for every i, and ask whether $dp[n][W]$ equals W. Equivalent: a 1-bit DP over feasibility.

Partition equal subset sum. Given $a_1, \ldots, a_n$, decide if some subset sums to $S/2$, where $S = \sum a_i$. Run subset-sum DP with $W = S/2$.

Count of subsets with target sum. Replace max with $+$, initial value 0, $dp[0][0] = 1$, and a mod.

Target sum with $\pm$ signs. Given $a_1, \ldots, a_n$, assign $\pm$ to each so that the signed sum equals T. Reduce to subset sum: if P is the sum of the $+$ items and N the sum of the $-$ items, then $P - N = T$ and $P + N = S$, hence $P = (S + T)/2$. So count subsets of $\{a_i\}$ summing to $(S + T)/2$.

Number partitioning. As above, ask for the number of ways to partition into two equal-sum halves.

3.3 Unbounded knapsack

3.3.1 Canonical problem

Same setup, but now each item can be used *any number of times*. The textbook version is the coin-change problem: count the number of ways to make amount W using an unlimited supply of coins $c_1, \ldots, c_n$.

3.3.2 Recurrence

The state $dp[i][c]$ means the same as before, but the "take item i" branch does not consume the item:

$$dp[i][c] = \max(dp[i-1][c], \;\; dp[i][c - w_i] + v_i) \tag{3.2}$$

The crucial difference is that the take-branch reads from *row* i (same item, smaller capacity) rather than from row $i - 1$.

3.3.3 C++17 implementation (coin change: count of ways)

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1'000'000'007;

int main(){
    int n; ll X;
    cin >> n >> X;
    vector<ll> c(n);
    for(auto &x : c) cin >> x;
    vector<ll> dp(X+1, 0);
    dp[0] = 1;                          // one way to make amount 0: take nothing
    for(int i=0;i<n;i++){
        for(ll x = c[i]; x <= X; x++){  // LOW to HIGH: reuse same coin
            dp[x] = (dp[x] + dp[x - c[i]]) % MOD;
        }
    }
    cout << dp[X] << "\n";
    return 0;
}
```

Warning 3.1: Inner loop order matters

For "count of ways to make X with coins" (where *order of coins does not matter*), the outer loop is over *coins* and the inner loop is over *amounts*. If you swap the loops — outer over amounts, inner over coins — you count *ordered* compositions, which is a different problem. Always check whether the problem counts ordered or unordered combinations.

3.4 Bounded knapsack

3.4.1 Canonical problem

Each item i has a maximum usage count m_i (a finite bound, but potentially large). Naively we could split item i into m_i identical 0/1 items, but that blows up to $\sum m_i$ items.

3.4.2 The binary-grouping trick

To reduce the number of 0/1 items from $\sum m_i$ to $\sum_i \lceil \log_2 m_i \rceil$, represent item i with usage bound m_i as a collection of 0/1 items whose multiplicities are $1, 2, 4, \ldots, 2^k, r$ where $2^k + 1 + \cdots + 2^k \le m_i$ and the remainder $r = m_i - (2^{k+1} - 1)$. Any integer in $[0, m_i]$ can be written as a sum of a subset of these groupings — the standard binary representation argument — so the resulting 0/1 knapsack computes the same maximum as the original bounded knapsack.

```
// Build the equivalent 0/1 item list from a bounded knapsack spec.
vector<ll> w01, v01;
for(int i=0;i<n;i++){
    ll m = m_i, k = 1;
    while(m > 0){
        ll take = min(k, m);
        w01.push_back(take * w[i]);
        v01.push_back(take * v[i]);
        m -= take;
        k <<= 1;
    }
}
// Then run the standard 0/1 knapsack on (w01, v01).
```

3.4.3 Monotonic-queue optimisation (advanced)

For a fully $O(nW)$ bounded knapsack (with $\sum m_i$ allowed to be arbitrary), we can use a sliding-window maximum on each residue class modulo w_i. See Section 14.3.2 for the full derivation; the binary-grouping trick above is usually sufficient in contests.

3.5 Subset sum variants in detail

3.5.1 Subset sum (boolean feasibility)

Given $a_1, \ldots, a_n$ and a target T, decide whether some subset sums to T. The 0/1 knapsack reduction: set $w_i = a_i$, $v_i = a_i$, capacity T, and ask whether $dp[n][T] = T$. The boolean version below is faster and uses 1 bit per state:

```
bitset<100001> dp;
dp[0] = 1;
for(int i=0;i<n;i++)
    dp |= (dp << a[i]);     // elegant: shift the reachable set by a_i
cout << (dp[T] ? "YES" : "NO") << "\n";
```

The `bitset` trick reduces time by a factor of ~ 64 compared to the plain boolean DP, often making $n \cdot W \sim 10^8$ tractable in 1 second.

3.5.2 Partition equal subset sum (Leetcode 416)

Given $a_1, \ldots, a_n$, partition into two subsets of equal sum iff $S = \sum a_i$ is even and there is a subset summing to $S/2$. Run the boolean subset-sum DP above with $T = S/2$.

3.5.3 Last-stone-weight / minimum subset-sum difference

Given $a_1, \ldots, a_n$, split into two subsets whose sums differ as little as possible. Equivalently, find a subset whose sum is as close to $S/2$ as possible. Use the subset-sum boolean DP and scan the largest reachable $T \le S/2$.

3.5.4 Count of subsets with sum exactly T

Replace the `OR` by a `+` modulo $10^9 + 7$:

```
vector<ll> dp(T+1, 0);
dp[0] = 1;
for(int i=0;i<n;i++)
    for(int t = T; t >= a[i]; t--)
        dp[t] = (dp[t] + dp[t - a[i]]) % MOD;
```

3.6 Two-dimensional knapsack (cost & weight)

If each item has both a *weight* and a *volume*, and the knapsack has both a weight capacity W and a volume capacity V, the state becomes (i, c, v):

$$dp[i][c][v] = \max(dp[i-1][c][v], \quad dp[i-1][c-w_i][v-v_i] + val_i) \tag{3.3}$$

$O(nWV)$ time. Watch the constraint sizes: with $n, W, V \le 200$ this is 8×10^6 states, comfortably in time; with $W, V \le 2000$ it is 8×10^9, too slow.

3.7 Problem set

P1. AtCoder DP Contest, Problem D (Knapsack 1). $n \le 100$ items, $W \le 10^5$, $v_i \le 10^9$. Find the maximum total value. *Hint:* straightforward 0/1 knapsack with 64-bit values.

P2. AtCoder DP Contest, Problem E (Knapsack 2). $n \le 100$ items, $W \le 10^9$, $v_i \le 10^3$, $\sum v_i \le 10^5$. Find the maximum value. *Hint:* swap the state axes. State: $dp[v]$ = minimum weight to achieve value $\ge v$. Iterate items, $dp[v] = \min(dp[v], dp[v-v_i] + w_i)$. Answer: largest v with $dp[v] \le W$.

P3. Leetcode 416, Partition Equal Subset Sum. $a_i \le 100$, $n \le 200$. Decide partition. *Solution:*

```
bool canPartition(vector<int>& a){
    int s = accumulate(a.begin(), a.end(), 0);
    if(s & 1) return false;
    int T = s/2;
    bitset<20001> dp; dp[0] = 1;
    for(int x : a) dp |= (dp << x);
    return dp[T];
}
```

P4. Leetcode 494, Target Sum. $a_i \le 1000$, $n \le 20$, target $T \in [-1000, 1000]$. Count the number of $\pm$ assignments. *Solution:* reduce to subset sum as in Section 3.2.7.

P5. Codeforces 687C, The Values You Can Make. $n \le 500$, $a_i \le 500$, target $k \le 500$. Use a subset of the a_i summing to k; what subset-sums can the chosen subset itself make? *Hint:* 2-D subset-sum DP. State: $dp[s][t]$ = true iff we can use some subset summing to s and that subset can be further partitioned to sum to t. Transition: for each a_i, $dp[s + a_i][t]| = dp[s][t]$ and $dp[s + a_i][t + a_i]| = dp[s][t]$.

P6. CSES, Coin Combinations I (Ordered). $n \le 100$ coins, $X \le 10^6$, count ordered ways. *Solution:* outer loop over x (amounts), inner over coins.

P7. CSES, Coin Combinations II (Unordered). Same input, count unordered ways. *Solution:* outer loop over coins, inner over x.

P8. SPOJ ABCDEF. Find sextuples (a, b, c, d, e, f) with $a \cdot b + c = d \cdot e + f$ from a set of $n \le 100$ integers. *Hint:* meet-in-the-middle. Compute all values of $a \cdot b + c$ and of $-(d \cdot e + f)$, sort, count matching pairs. Knapsack-style but with $O(n^3)$ enumerated values rather than $O(nW)$ states.

3.8 Common pitfalls

- **Integer overflow.** Values can be 10^9, counts up to 10^{18}. Use `long long` everywhere unless you are absolutely certain $v_i \le 10^4$.
- **Loop order for counting versions.** Always check whether "order matters" in the count.
- **Off-by-one in capacity.** Capacity W means weights $\le W$ are accepted; the array has $W + 1$ entries indexed $0..W$.
- **Pseudo-polynomiality.** If $W > 10^7$ the DP will not fit. Look for an alternate state (AtCoder DP E: swap axes to value).
- **Bitset shift overflow.** When using `bitset<N>` for subset sum, make sure $N > \max a_i$ (per-shift sum).

Chapter 4

The LIS / LCS Family

4.1 The unifying idea: DP on indices into a sequence

The LIS/LCS family consists of DPs whose state is a tuple of indices into one or two sequences. The transition typically involves picking the "previous chosen index" and growing the subsequence by one element.

The two canonical problems are:

- **Longest Increasing Subsequence (LIS)** of $a[1..n]$: the longest subsequence whose values are strictly increasing.
- **Longest Common Subsequence (LCS)** of $a[1..n]$ and $b[1..m]$: the longest sequence that is a subsequence of both.

Both are quadratically expensive in their naive form, and both admit clever $O(n \log n)$ or $O(nm$-style) improvements that you should memorise once and reuse forever.

4.2 Longest Increasing Subsequence

4.2.1 Canonical problem

Given a sequence $a_1, a_2, \ldots, a_n$, find the length of the longest strictly increasing subsequence. (Replace "strictly" by "non-decreasing" or "decreasing" with trivial modifications.)

4.2.2 $O(n^2)$ DP

$$dp[i] = 1 + \max\{dp[j] : j < i,\ a_j < a_i\} \quad \text{(or 1 if no such } j\text{)} \tag{4.1}$$

State: "length of the longest increasing subsequence ending at index i". Answer: $\max_i dp[i]$.

```
int n = a.size();
vector<int> dp(n, 1);
for(int i=0;i<n;i++)
    for(int j=0;j<i;j++)
        if(a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
int ans = *max_element(dp.begin(), dp.end());
```

$O(n^2)$ time. Fine for $n \le 5000$; too slow for $n \ge 10^5$.

4.2.3 $O(n \log n)$ via patience sorting

The classical improvement stores, for every length ℓ, the smallest possible tail value $t[\ell]$ of an increasing subsequence of length ℓ seen so far. The array t is non-decreasing by construction, which allows binary search.

```cpp
int n = a.size();
vector<int> t;          // t[l] = smallest tail value of an increasing
                        //        subsequence of length l+1 seen so far
for(int x : a){
    // Strictly increasing: lower_bound; non-decreasing: upper_bound.
    auto it = lower_bound(t.begin(), t.end(), x);
    if(it == t.end()) t.push_back(x);
    else              *it = x;
}
int lis = t.size();
```

Why it works. For each x, the position at which x extends the longest subsequence is exactly the position of the first tail $t[\ell] \geq x$. We replace $t[\ell]$ with x, which is no worse (because $x \leq t[\ell]$) and might enable better future extensions.

Reconstructing the LIS. To reconstruct the actual subsequence, store the index of the predecessor at each step and walk backwards. See the `parent` array in the snippet below.

```cpp
vector<int> t;            // tail values
vector<int> t_idx;        // index in `a` of each tail value
vector<int> parent(n, -1);
for(int i=0;i<n;i++){
    auto it = lower_bound(t.begin(), t.end(), a[i]);
    int pos = it - t.begin();
    if(it == t.end()){ t.push_back(a[i]); t_idx.push_back(i); }
    else            { *it = a[i];          t_idx[pos] = i; }
    parent[i] = (pos > 0) ? t_idx[pos-1] : -1;
}
// Reconstruct
vector<int> seq;
for(int cur = t_idx.back(); cur != -1; cur = parent[cur])
    seq.push_back(a[cur]);
reverse(seq.begin(), seq.end());
```

Tip 4.1: Patience sorting is also patience sorting

The algorithm above is also a way to compute the patience-sorting partition of the sequence into piles, and Dilworth's theorem gives that the number of piles equals the length of the LIS. This is a useful sanity check: write the patience partition, count piles, and verify against the LIS value.

4.2.4 Variants

- **Longest Decreasing Subsequence (LDS).** Reverse the sequence and run LIS, or negate the values.
- **Longest Bitonic Subsequence.** A sequence that first increases then decreases. Compute lis_i from the left and lds_i from the right; the answer is $\max_i(\text{lis}_i + \text{lds}_i - 1)$.
- **Longest Increasing Subsequence with at most k replacements.** Use a segment tree over value positions.
- **Count the number of LISs.** Use the Fenwick tree variant: store pairs $(\text{length}, \text{count})$ indexed by value, with $+$ as the merge operator.

4.3 Longest Common Subsequence

4.3.1 Canonical problem

Given strings a of length n and b of length m, find the length of the longest common subsequence.

4.3.2 $O(nm)$ DP

State $dp[i][j]$ = "length of the LCS of $a[1..i]$ and $b[1..j]$". Transition:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } a_i = b_j, \\ \max(dp[i-1][j],\ dp[i][j-1]) & \text{otherwise.} \end{cases} \tag{4.2}$$

Base: $dp[0][j] = dp[i][0] = 0$. Answer: $dp[n][m]$.

```
int n = a.size(), m = b.size();
vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
for(int i=1;i<=n;i++)
    for(int j=1;j<=m;j++){
        if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
        else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
    }
int ans = dp[n][m];
```

4.3.3 Reconstruction

Walk back from (n, m): if $a_i = b_j$, prepend a_i to the answer and move to $(i-1, j-1)$; otherwise move to whichever of $(i-1, j)$ and $(i, j-1)$ has the larger DP value.

4.3.4 Memory reduction: Hirschberg's algorithm

Naive $O(nm)$ memory is too large for $n, m \geq 10^4$. Hirschberg's algorithm computes the LCS in $O(nm)$ time but only $O(\min(n, m))$ memory by recursively splitting at the optimal middle column. The implementation is delicate; many contests prefer the simpler *two-rows* trick when $O(\min(n, m))$ memory suffices for length only (no reconstruction):

```
// Two-row trick: dp[i][j] only depends on row i-1 and the current row.
vector<vector<int>> dp(2, vector<int>(m+1, 0));
for(int i=1;i<=n;i++){
    int cur = i & 1, prev = cur ^ 1;
    for(int j=1;j<=m;j++){
        if(a[i-1] == b[j-1]) dp[cur][j] = dp[prev][j-1] + 1;
        else dp[cur][j] = max(dp[prev][j], dp[cur][j-1]);
    }
}
int ans = dp[n & 1][m];
```

4.3.5 $O((n+m)\log n)$ for restricted alphabets (Hunt-Szymanski)

If one of the strings is much shorter than the other and the alphabet is small, the Hunt-Szymanski algorithm runs in $O((|n| + |m|)\log |n|)$ where $n \leq m$ by reducing to LIS on the sequence of matching positions. Implementations are rare in contests; usually $O(nm)$ with the two-row trick is enough.

4.3.6 Variants

- **Edit distance (Levenshtein).** Allowed operations: insert, delete, substitute, each cost 1. State $dp[i][j]$ = "minimum number of operations to transform $a[1..i]$ into $b[1..j]$".

- **Longest Common Substring.** *Subsequence* → *substring* (contiguous). State $dp[i][j]$ = "length of the longest common *suffix* of $a[1..i]$ and $b[1..j]$ that ends at positions i and j". Transition: $dp[i][j] = dp[i-1][j-1] + 1$ if $a_i = b_j$, else 0. Answer: $\max dp$.
- **Shortest Common Supersequence.** Length $= n + m -$ LCS.
- **Number of common subsequences.** Use the recurrence

$$dp[i][j] = dp[i-1][j] + dp[i][j-1] - dp[i-1][j-1] + [a_i = b_j] \cdot dp[i-1][j-1].$$

- **LCS with restricted operations.** Allow only deletion; only insertion; weighted costs; etc.

4.4 Edit distance in detail

$$dp[i][j] = \begin{cases} dp[i-1][j-1] & \text{if } a_i = b_j, \\ 1 + \min\left(dp[i-1][j-1],\ dp[i-1][j],\ dp[i][j-1]\right) & \text{otherwise.} \end{cases} \tag{4.3}$$

The three options correspond to substitute, delete-from-a, and insert-into-a. Weighted edit distance replaces 1 by the appropriate cost. With custom costs, allow arbitrary non-negative integer costs and the same DP.

```
int n = a.size(), m = b.size();
vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
for(int i=0;i<=n;i++) dp[i][0] = i;     // delete all of a
for(int j=0;j<=m;j++) dp[0][j] = j;     // insert all of b
for(int i=1;i<=n;i++)
    for(int j=1;j<=m;j++){
        if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1];
        else dp[i][j] = 1 + min({dp[i-1][j-1], dp[i-1][j], dp[i][j-1]});
    }
int ans = dp[n][m];
```

4.5 Problem set

P1. CSES, Increasing Subsequence. $n \le 2 \cdot 10^5$. Find LIS length. *Solution:* patience sort, $O(n \log n)$.

P2. CSES, Projects. $n \le 2 \cdot 10^5$ projects with start/end times and rewards. Choose a subset of non-overlapping projects maximising total reward. *Hint:* sort by end time, coordinate-compress the start time, and use a Fenwick tree of maximums. The DP is $dp[i]$ = "maximum reward using only projects among the first i (in end-time order)"; the transition looks up the best $dp[j]$ for projects ending before project i starts.

P3. AtCoder DP Contest, Problem G (Longest Path). This is actually a DAG longest-path DP, but the same "DP on indices" flavour applies. See Chapter 7.

P4. AtCoder DP Contest, Problem F (LCS). Two strings of length ≤ 3000 each. Find the longest common subsequence *and* one of its representatives. *Solution:* $O(nm)$ DP plus reconstruction.

P5. AtCoder DP Contest, Problem E (Knapsack 2). Already covered; included here as a reminder that "swap axes when one is too large" is a recurring LIS/LCS-style trick.

P6. Codeforces 10D, Longest Common Increasing Subsequence. Given a and b, find the longest sequence that is simultaneously an increasing subsequence of a and a subsequence of b. *Hint:* combine LIS-style updates into the LCS DP. State $dp[i][j]$ = "length of LCIS ending at $a_i = b_j$"; maintain the best dp value as j moves for fixed i.

P7. Leetcode 1143, Longest Common Subsequence. $n, m \leq 1000$. Standard LCS. *Solution:* the code above.

P8. Leetcode 72, Edit Distance. $n, m \leq 500$. *Solution:* the code in Section 4.4.

P9. Codeforces 1446C, Sequence Pair Weight. Count the sum over all pairs of indices into a sequence of the number of common subsequences. *Hint:* DP counts the number of common subsequences between each pair of prefixes.

4.6 Common pitfalls

- **Strict vs non-strict.** Use `lower_bound` for strictly increasing, `upper_bound` for non-decreasing. Getting this wrong is a one-line bug that fails only on inputs with duplicates.
- **1-indexing for LCS.** The DP table has a row and column for "empty prefix"; this is the most common source of off-by-one bugs.
- **Reconstruction walking the wrong neighbour.** When reconstructing, in case of ties prefer the diagonal move (so that the output character is emitted at the correct moment).
- **Hirschberg on the wrong axis.** Always split along the longer dimension; the recursion depth is bounded by the shorter dimension.

Chapter 5

Interval DP

5.1 The unifying idea

An *interval DP* is one whose state is a contiguous sub-interval $[l, r]$ of an array. The transition combines two smaller intervals $[l, k]$ and $[k+1, r]$ (for some cut point k) into the larger interval $[l, r]$. The state space is $O(n^2)$ and the transition is $O(n)$, giving $O(n^3)$ overall — so interval DP is the right tool whenever the problem asks for "the best order to combine contiguous pieces of an array", with $n \leq 100$ (or so).

The canonical examples:

- **Matrix Chain Multiplication (MCM).** Best parenthesisation of a chain of matrix multiplications.
- **Burst Balloons.** Best order to remove elements, where the cost of removing a_i depends on its current neighbours.
- **Merging Stones.** Best order to merge adjacent piles, with a per-merge cost.
- **Optimal Binary Search Tree.** Best shape for a BST given query frequencies.
- **Bracket Sequence** problems. Find a way to pair brackets in $O(n^3)$.

5.2 General interval DP template

1. Define $dp[l][r]$ = "optimal value on the sub-array $a[l..r]$".
2. Iterate *by length* $len = 2, 3, \ldots, n$. For each len, iterate the start $l = 1, 2, \ldots, n - len + 1$; let $r = l + len - 1$. For each (l, r), try every cut $k \in [l, r-1]$ and combine $dp[l][k]$ with $dp[k+1][r]$.
3. Store the best combination; the answer is $dp[1][n]$.

```
// Generic O(n^3) interval DP skeleton.
vector<vector<long long>> dp(n+1, vector<long long>(n+1, 0));
for(int len = 2; len <= n; len++){
    for(int l = 1; l + len - 1 <= n; l++){
        int r = l + len - 1;
        dp[l][r] = INF;                    // sentinel for min DP
        for(int k = l; k < r; k++){
            long long cand = combine(dp[l][k], dp[k+1][r]) + cost(l, r, k);
            dp[l][r] = min(dp[l][r], cand);
        }
    }
}
```

The `cost(l, r, k)` function depends on the problem: for MCM it is $p_{l-1} \cdot p_k \cdot p_r$; for merging stones it is prefix_sum$[r]$ − prefix_sum$[l-1]$; for burst balloons it is $a[l-1] \cdot a[k] \cdot a[r+1]$.

Tip 5.1: Length-first iteration

Always iterate by *length* (small to large), never by starting position. The length-first loop guarantees that whenever you compute $dp[l][r]$, both $dp[l][k]$ and $dp[k+1][r]$ are already computed (because their lengths are strictly smaller than len).

5.3 Matrix Chain Multiplication

5.3.1 Canonical problem

Given matrices $M_1, M_2, \ldots, M_n$ where M_i has dimensions $p_{i-1} \times p_i$, find the parenthesisation that minimises the total number of scalar multiplications.

5.3.2 Recurrence

If we split the chain between k and $k+1$, the cost is

$$\text{cost}(l, r, k) = dp[l][k] + dp[k+1][r] + p_{l-1} \cdot p_k \cdot p_r.$$

The first two terms are the costs of multiplying each half internally; the third is the cost of multiplying the two resulting matrices of sizes $p_{l-1} \times p_k$ and $p_k \times p_r$.

$$\boxed{dp[l][r] = \min_{k \in [l, r-1]} \Big(dp[l][k] + dp[k+1][r] + p_{l-1}\, p_k\, p_r \Big)} \tag{5.1}$$

Base: $dp[i][i] = 0$ (single matrix, no multiplication). Answer: $dp[1][n]$.

5.3.3 C++17 implementation

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main(){
    int n; cin >> n;
    vector<ll> p(n+1);
    for(int i=0;i<=n;i++) cin >> p[i];
    vector<vector<ll>> dp(n+1, vector<ll>(n+1, 0));
    for(int len = 2; len <= n; len++){
        for(int l = 1; l + len - 1 <= n; l++){
            int r = l + len - 1;
            dp[l][r] = LLONG_MAX;
            for(int k = l; k < r; k++){
                ll cand = dp[l][k] + dp[k+1][r] + p[l-1]*p[k]*p[r];
                dp[l][r] = min(dp[l][r], cand);
            }
        }
    }
    cout << dp[1][n] << "\n";
    return 0;
}
```

5.3.4 Knuth optimisation

MCM satisfies the *quadrangle inequality* and admits Knuth's optimisation, which restricts the cut search to $K[l, r-1] \le k \le K[l+1, r]$, reducing the total work to $O(n^2)$. See Section 14.5.

5.4 Burst Balloons

5.4.1 Canonical problem

Given n balloons with values $a_1, \ldots, a_n$, plus sentinels $a_0 = a_{n+1} = 1$. When you burst balloon i, you gain $a_{i-1} \cdot a_i \cdot a_{i+1}$, then a_{i-1} and a_{i+1} become adjacent. Find the maximum total gain.

5.4.2 Recurrence

This is a classic interval DP that *thinks backwards*: instead of "which balloon to burst first", ask "which balloon is burst last in the interval $[l, r]$". The last balloon burst inside $[l, r]$ has its neighbours being $a[l-1]$ and $a[r+1]$ (sentinels), because everything else has already been removed. If the last balloon is k, the gain is

$$dp[l][r] = \max_{k \in [l,r]} \big(dp[l][k-1] + dp[k+1][r] + a[l-1] \cdot a[k] \cdot a[r+1] \big).$$

Base: $dp[l][r] = 0$ if $l > r$. Answer: $dp[1][n]$.

5.4.3 C++17 implementation

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main(){
    int n; cin >> n;
    vector<ll> a(n+2);
    a[0] = a[n+1] = 1;
    for(int i=1;i<=n;i++) cin >> a[i];
    vector<vector<ll>> dp(n+2, vector<ll>(n+2, 0));
    for(int len = 1; len <= n; len++){
        for(int l = 1; l + len - 1 <= n; l++){
            int r = l + len - 1;
            for(int k = l; k <= r; k++){
                ll cand = dp[l][k-1] + dp[k+1][r] + a[l-1]*a[k]*a[r+1];
                dp[l][r] = max(dp[l][r], cand);
            }
        }
    }
    cout << dp[1][n] << "\n";
    return 0;
}
```

Tip 5.2: Why the "last burst" trick?

If we tried to enumerate "first balloon to burst", the cost depends on the *current* neighbours, which depend on *all previous bursts*, not just on the interval $[l, r]$. The trick is to enumerate "last balloon to burst inside $[l, r]$" instead — because the last balloon's neighbours are exactly the sentinels $a[l-1]$ and $a[r+1]$, which are independent of any internal order.

5.5 Merging stones (Yandex / Leetcode 1000)

5.5.1 Canonical problem

n piles of stones with sizes $a_1, \ldots, a_n$. You may merge *exactly* k *consecutive* piles into one pile at a cost of the sum of their sizes. Find the minimum cost to reduce to one pile, or report impossible.

5.5.2 Recurrence (3-D)

State: $dp[l][r][m]$ = "minimum cost to merge piles $a[l..r]$ into exactly m piles". The transition:

- $m = 1$: pick a cut $t \in [l, r-1]$ such that $a[l..t]$ can be merged into $k-1$ piles and $a[t+1..r]$ can be merged into 1 pile; then merge these k piles into 1 at a cost of $\text{sum}[l..r]$:

$$dp[l][r][1] = \min_t dp[l][t][k-1] + dp[t+1][r][1] + \text{sum}[l..r].$$

  This needs $(r - l + 1)$ to be a multiple of $(k - 1)$.

- $m > 1$: pick a cut t:

$$dp[l][r][m] = \min_t dp[l][t][1] + dp[t+1][r][m-1].$$

Time: $O(n^3 \cdot k)$.

5.5.3 Simpler variant: merge *any* two adjacent piles (Huffman-style)

If the rule is "merge any two adjacent piles, cost = their sum", then the problem is just a 2-D interval DP with $cost(l, r, k) = \text{sum}[l..r]$ which is constant in k:

$$dp[l][r] = \min_{k \in [l, r-1]} (dp[l][k] + dp[k+1][r]) + \text{sum}[l..r]. \tag{5.2}$$

5.5.4 C++17 (simpler variant)

```
int n; cin >> n;
vector<ll> a(n+1);
for(int i=1;i<=n;i++) cin >> a[i];
vector<ll> pref(n+1, 0);
for(int i=1;i<=n;i++) pref[i] = pref[i-1] + a[i];
auto sum = [&](int l, int r){ return pref[r] - pref[l-1]; };
vector<vector<ll>> dp(n+1, vector<ll>(n+1, 0));
for(int len = 2; len <= n; len++){
    for(int l = 1; l + len - 1 <= n; l++){
        int r = l + len - 1;
        dp[l][r] = LLONG_MAX;
        for(int k = l; k < r; k++)
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r]);
        dp[l][r] += sum(l, r);
    }
}
cout << dp[1][n] << "\n";
```

This naive $O(n^3)$ solution can be optimised to $O(n^2)$ via Knuth optimisation (Section 14.5) because the cost function $\text{sum}[l..r]$ satisfies the quadrangle inequality.

5.6 Optimal Binary Search Tree

5.6.1 Canonical problem

Given keys $k_1 < \cdots < k_n$ with query frequencies $f_1, \ldots, f_n$ (also for "dummy" keys between them, with frequencies $d_0, \ldots, d_n$), find a BST minimising the expected depth of a query.

5.6.2 Recurrence

State $dp[l][r]$ = "minimum expected depth on the sub-problem with keys $k_l, \dots, k_r$ and dummies $d_{l-1}, \dots, d_r$".

$$dp[l][r] = \min_{m=l}^{r} \Big(dp[l][m-1] + dp[m+1][r] + W(l,r) \Big) \tag{5.3}$$

where $W(l,r) = \sum_{i=l}^{r} f_i + \sum_{i=l-1}^{r} d_i$ is the sum of frequencies in the subproblem (the root adds 1 to every depth). Base: $dp[l][r] = d_l$ if $r = l-1$. $O(n^3)$ naive; $O(n^2)$ with Knuth.

5.7 Bracket sequences

5.7.1 Canonical problem

Given a bracket string of length n, find the maximum number of correctly matched pairs in a subsequence.

5.7.2 Recurrence

State $dp[l][r]$ = "maximum matched pairs in $s[l..r]$". Transition:

- If $s[l]$ and $s[r]$ match (one open, one close in the right order), we can pair them and add $1 + dp[l+1][r-1]$.
- Otherwise, split: $\max_k dp[l][k] + dp[k+1][r]$.

$$dp[l][r] = \max\left(dp[l+1][r-1] + [s_l \text{ matches } s_r],\ \max_{k \in [l, r-1]} dp[l][k] + dp[k+1][r] \right). \tag{5.4}$$

$O(n^3)$. For counting regular bracket sequences of length $2n$ with additional constraints, see Section 5.7.3.

5.7.3 Catalan-style counting of bracket sequences

The number of regular bracket sequences of length $2n$ is the Catalan number $C_n = \frac{1}{n+1}\binom{2n}{n}$. Variants:

- Count regular sequences with k opening brackets at the start.
- Count sequences avoiding some substring T: use the Aho-Corasick automaton of forbidden words as state (see Chapter 12).

5.8 Problem set

P1. CSES, Matrix Chain Multiplication. $n \le 500$ matrices. *Solution:* the MCM code above; consider Knuth optimisation if you need $n \le 1000$.

P2. Leetcode 312, Burst Balloons. $n \le 500$. *Solution:* the burst-balloons code above.

P3. Leetcode 1000, Minimum Cost to Merge Stones. $n \le 30$, $k \le 30$. *Solution:* the 3-D interval DP of Section 5.5.

P4. Leetcode 375, Guess Number Higher or Lower II. $n \le 300$. Minimax over the worst-case cost of guessing a number in $[1, n]$. *Hint:* $dp[l][r] = \min_k \max(dp[l][k-1], dp[k+1][r]) + k$ — a minimax interval DP. Same skeleton; replace `max` on the combination step with `min(max...)`.

P5. SPOJ MKEYS, Matrix Chain Multiplication. Same as CSES but with multiple test cases.

P6. Codeforces 1132F, Clear the String. $n \le 500$. You may delete any contiguous palindromic substring in one operation; find the minimum number of operations to delete the whole string. *Hint:* interval DP with state $dp[l][r]$ = min operations to delete $s[l..r]$. Transition: try to delete $s[r]$ together with some $s[k]$ where $s[k] = s[r]$, in which case $dp[l][r] = \min(dp[l][r], dp[l][k] + dp[k+1][r-1])$; also handle $dp[l][r] = \min(dp[l][r], dp[l][r-1] + 1)$.

P7. Codeforces 607B, Zuma. $n \le 500$. Remove palindromic substrings in minimum operations. Similar to P6.

P8. Codeforces 1114D, Flood Fill. $n \le 5000$. Adjacent cells of the same colour form a connected component; you can repaint one component at a time. Find minimum operations to make the whole array one colour. *Hint:* compress equal adjacent values into blocks, then run a 1-D interval DP $dp[l][r]$ = min operations to make blocks $l..r$ uniform.

5.9 Common pitfalls

- **Iteration order.** Always by length-first. A start-first loop silently uses uninitialised values.
- **Out-of-range sentinels.** Burst-balloons needs $a[0] = a[n+1] = 1$ *and* the DP table indexed up to $n+1$.
- **Memory for $n = 500$.** The $n \times n$ table of `long long` uses 2 MB; fine. For $n = 5000$ you need 200 MB and likely MLE; consider splitting the DP into the half-arrays of Hirschberg-style.
- **Initialisation for min-DP.** Use `LLONG_MAX` or `numeric_limits<ll>::max()` but be careful not to add to it (overflow). Use `INF = 1e18` (an explicit large constant) when sums are involved.

Chapter 6

Tree DP

6.1 The unifying idea

A *tree DP* is one whose state is a subtree rooted at a vertex u. The transition merges the DP values of the children of u. Tree DP is the natural form when the input is a tree and we are asked about a property that is defined "locally" on subtrees — diameters, maximum independent sets, matchings, "sum of distances", rerooting problems, and tree knapsack.

Two complications make tree DP harder than linear DP:

1. **Order of computation.** We must process vertices in post-order (children before parents). This means either an explicit recursion from the root, or a topological sort by depth descending.
2. **Merging children** can be expensive if not done carefully. The naive "for each child, update parent" approach is $O(\sum \text{subtree sizes})$ which can be $O(n^2)$ on a path graph.

6.2 Setting up the recursion

The cleanest pattern in C++ is an explicit recursion from the root, with a `visited` marker to handle the parent edge:

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int n;
vector<vector<int>> adj;
vector<ll> dp;     // dp[u] = some subtree property of u

void dfs(int u, int p){
    dp[u] = 0;                          // base
    for(int v : adj[u]) if(v != p){
        dfs(v, u);
        // merge dp[v] into dp[u] here
        dp[u] += dp[v];                 // or max, or other
    }
    // apply the ``local'' contribution of u itself, if any
    dp[u] += 1;                         // example: count nodes
}

int main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);
    cin >> n;
    adj.assign(n, {});
    for(int i=0;i<n-1;i++){
        int u,v; cin >> u >> v; --u; --v;
        adj[u].push_back(v);
```

```
        adj[v].push_back(u);
    }
    dp.assign(n, 0);
    dfs(0, -1);
    cout << dp[0] << "\n";
    return 0;
}
```

Tip 6.1: Recursion depth

For $n \leq 10^5$, deep recursion can blow the stack (Linux default is 8 MB $\approx 10^5$ stack frames for trivial recursion). Always either (a) increase the stack with `ulimit -s unlimited` before running, (b) rewrite the DFS iteratively, or (c) use BFS for the order. The `ulimit` trick is contest-legal and easiest.

6.3 Tree diameter

6.3.1 Canonical problem

Find the length (in edges) of the longest path in a tree.

6.3.2 Two approaches

1. **Two BFS / DFS.** Run a BFS from any vertex u; let v be the farthest. Run a BFS from v; let w be the farthest; then $d(v, w)$ is the diameter. $O(n)$, no DP needed. Works only on *unweighted* trees.
2. **Tree DP.** State $dp[u]$ = "maximum distance from u into its subtree" ("height"). The diameter through u is the sum of the two largest child heights, plus 2 (for the two edges into the children). The global diameter is the max over all u. $O(n)$, also works on weighted trees.

```
ll ans = 0;
vector<ll> h(n, 0);            // h[u] = max distance from u into subtree

function<void(int,int)> dfs = [&](int u, int p){
    ll m1 = 0, m2 = 0;          // two largest child heights
    for(int v : adj[u]) if(v != p){
        dfs(v, u);
        ll cand = h[v] + 1;     // edge u--v
        if(cand > m1){ m2 = m1; m1 = cand; }
        else if(cand > m2) m2 = cand;
    }
    h[u] = m1;
    ans = max(ans, m1 + m2);
};
dfs(0, -1);
```

6.4 Maximum independent set on a tree

6.4.1 Canonical problem

Choose a subset of vertices, no two of which are adjacent, maximising the total weight.

6.4.2 Recurrence

State $dp[u][0/1]$ = "best value in the subtree of u, where the index 0/1 says whether u itself is excluded or included".

$$dp[u][0] = \sum_{v \in \text{child}(u)} \max(dp[v][0], dp[v][1]) \tag{6.1}$$

$$dp[u][1] = w_u + \sum_{v \in \text{child}(u)} dp[v][0] \tag{6.2}$$

Base case: $dp[u][0] = 0$, $dp[u][1] = w_u$ at a leaf. Answer: $\max(dp[r][0], dp[r][1])$ where r is the root.

```
vector<array<ll,2>> dp(n);
function<void(int,int)> dfs = [&](int u, int p){
    dp[u][0] = 0;
    dp[u][1] = w[u];
    for(int v : adj[u]) if(v != p){
        dfs(v, u);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
};
dfs(0, -1);
ll ans = max(dp[0][0], dp[0][1]);
```

6.5 Maximum matching on a tree

6.5.1 Canonical problem

Find the size of the maximum matching (set of pairwise non-adjacent edges) of a tree.

6.5.2 Recurrence

State $dp[u][0/1]$ = "max matching size in the subtree of u, where 0 means u is unmatched, 1 means u is matched to one of its children".

$$dp[u][0] = \sum_{v \in \text{child}(u)} \max(dp[v][0], dp[v][1]) \tag{6.3}$$

$$dp[u][1] = \max_{v \in \text{child}(u)} \Big(dp[v][0] + 1 + \sum_{w \neq v} dp[w][0] \Big) \tag{6.4}$$

The second line corresponds to picking the edge (u, v) for the matching. For each v, we add 1 for the edge and force every other child w to be in state 0 (since they cannot be matched to u).

A more compact and equivalent formulation: let $s = \sum_v \max(dp[v][0], dp[v][1])$. Then

$$dp[u][1] = s + \max_{v \in \text{child}(u)} \big(dp[v][0] - \max(dp[v][0], dp[v][1]) + 1 \big).$$

6.6 Rerooting technique

6.6.1 When to reroot

A problem is a rerooting problem if it asks for "the value of $f(u)$ for *every* u as root of the tree" (e.g. "sum of distances from every node to all other nodes"). The naive approach is to run the rooted DP n times, $O(n^2)$. The rerooting technique computes it in $O(n)$ total by reusing a forward pass and a backward pass.

6.6.2 Method

1. Pick an arbitrary root (say 0). Run a post-order DFS computing $dp[u]$ on the subtree of u.
2. Run a second (pre-order) DFS computing $up[u]$ = the answer for the rest of the tree if we treat u as root.
3. The final answer for u is the combination of $dp[u]$ (subtree of u) and $up[u]$ (everything else).

6.6.3 Canonical example: sum of distances from every node

```
vector<ll> sz(n), sd(n);          // sz[u]=subtree size, sd[u]=sum of distances in subtree
// First DFS: post-order.
function<void(int,int)> dfs1 = [&](int u, int p){
    sz[u] = 1; sd[u] = 0;
    for(int v : adj[u]) if(v != p){
        dfs1(v, u);
        sz[u] += sz[v];
        sd[u] += sd[v] + sz[v];   // every node in v's subtree gets 1 extra edge
    }
};
// Second DFS: pre-order, reroot.
vector<ll> ans(n);
function<void(int,int,ll)> dfs2 = [&](int u, int p, ll up){
    ans[u] = sd[u] + up;
    for(int v : adj[u]) if(v != p){
        // remove v's subtree from u and re-attach u as a child of v
        ll new_up = ans[u] - (sd[v] + sz[v]) + (n - sz[v]);
        dfs2(v, u, new_up);
    }
};
dfs1(0, -1);
dfs2(0, -1, 0);
```

Tip 6.2: Rerooting in words

Rerooting is a "peel off child v, push u (and its accumulated parent side) onto v as a new child" operation. Concretely: the contribution of v's original subtree is subtracted, and the contribution of "everything except v's subtree" is added as v's new parent edge.

6.6.4 Generalised rerooting

For problems where the merge operation is more complex (e.g. rerooting the maximum independent set), store for each u the prefix and suffix of contributions from its children's list, and use them to recompute the answer without v in $O(\log)$ or $O(1)$ per child.

6.7 Tree knapsack

6.7.1 Canonical problem

You are given a tree with n vertices. Each vertex u has a cost c_u and a value v_u. You may select a subset of vertices forming a *connected subtree* containing the root, with total cost $\leq W$. Find the maximum total value.

6.7.2 Recurrence

State $dp[u][w]$ = "max value, considering only the subtree of u and using total cost $\leq w$, with u *selected*". The transition is a knapsack over the children: for each child v we choose how much

weight to allocate to v's subtree, and we accumulate via the standard 0/1 knapsack merge:

```
// dp[u] is a vector of length W+1.
function<void(int,int)> dfs = [&](int u, int p){
    // Allocate enough space; later we will compress by in-place merge.
    // Initialise: only u is selected.
    fill(dp[u].begin(), dp[u].end(), -INF);
    dp[u][c[u]] = v[u];                  // include u itself
    for(int v : adj[u]) if(v != p){
        dfs(v, u);
        // Knapsack-style merge: pick some w' to allocate to v.
        vector<ll> ndp(W+1, -INF);
        for(int w1 = 0; w1 <= W; w1++) if(dp[u][w1] >= 0){
            for(int w2 = 0; w1 + w2 <= W; w2++) if(dp[v][w2] >= 0){
                ndp[w1+w2] = max(ndp[w1+w2], dp[u][w1] + dp[v][w2]);
            }
        }
        dp[u] = ndp;
    }
};
dfs(0, -1);
ll ans = *max_element(dp[0].begin(), dp[0].end());
```

The total time is $O(n \cdot W)$ because of the *small-to-large* trick on subtree sizes: when we merge child v into u, the inner loops together cost $\min(W, \mathrm{sz}(u)) \cdot \min(W, \mathrm{sz}(v))$, which sums to $O(nW)$ over the whole tree (the proof is identical to that of small-to-large merging).

Warning 6.1: Watch the DP table size

The $O(nW)$ bound assumes that you only allocate table entries up to the subtree size. If you allocate the full $W+1$ entries for every node and merge naively, the cost can become $O(n^2 W)$ on a path graph.

6.8 Centroid decomposition (sketch)

For "path query" problems on trees (e.g. count paths of length exactly k), the standard solution is to use *centroid decomposition*: recursively pick the centroid of the current tree (the vertex whose removal leaves subtrees of size $\leq n/2$), solve the problem "paths through the centroid" with a subtree DP, and recurse into the subtrees. This gives $O(n \log n)$ for many path problems. The DP at each centroid is a knapsack-style merge of subtree depth histograms. Full derivation: see the references at the end of the chapter.

6.9 Problem set

P1. CSES, Tree Diameter. $n \leq 2 \cdot 10^5$. *Solution:* two-BFS or tree DP.

P2. CSES, Tree Distances I/II. For each vertex, sum of distances to all other vertices. *Solution:* rerooting, as in Section 6.6.

P3. CSES, Company Queries I/II. LCA via binary lifting. Tree DP setup: `up[u][k]` = 2^k-th ancestor of u.

P4. CSES, Path Queries. $n \leq 2 \cdot 10^5$. Count paths of length ℓ for every $\ell \leq n$. *Hint:* centroid decomposition with knapsack-style subtree DP.

P5. AtCoder DP Contest, Problem P (Tree Knapsack). $n \le 5000$ tree, $W \le 5000$. *Solution:* the tree knapsack code above, $O(nW)$.

P6. Codeforces 1187E, Count Paths. $n \le 3 \cdot 10^5$. Count the number of good colourings for every root. *Hint:* rerooting.

P7. Codeforces 1387C, Counting Greedy. Special tree DP with custom merging.

P8. SPOJ PT07X, Vertex Cover. $n \le 10^5$. Minimum vertex cover of a tree. *Solution:* the maximum-independent-set code, swapping the role of 0/1 states (a vertex cover is the complement of an independent set).

6.10 Common pitfalls

- **Stack overflow.** For $n \ge 10^5$ on a path graph, `ulimit -s unlimited` or rewrite as iterative DFS.
- **Double-counting the parent.** Always pass the parent in the DFS call and skip it in the loop.
- **Rerooting algebra.** Rerooting formulas are subtle; derive them on a small example before coding.
- **Tree knapsack complexity.** Naive merge gives $O(n^2 W)$; use the small-to-large trick to get $O(nW)$.
- **Mod arithmetic for counting DPs.** Apply `%MOD` after every addition; forgetting this leads to overflow only on the largest tests.

Chapter 7

DAG and Path-Counting DP

7.1 The unifying idea

A *DAG DP* is one whose state is a vertex of a directed acyclic graph and whose transitions follow the edges. Equivalently, the DP is a topological-sort-driven relaxation of a value over the vertices of the DAG. Any problem whose state space is naturally indexed by positions in a graph that admits a topological ordering is, in this sense, a DAG DP.

This is one of the most under-recognised DP families: many problems labelled "counting paths" or "longest path in a graph" are DAG DPs in disguise.

7.2 Topological-sort-driven DP

7.2.1 Algorithm

1. Verify that the graph is a DAG (no directed cycle). For general directed graphs the problem may be NP-hard (e.g. longest path in a general graph is NP-hard).
2. Compute a topological ordering $v_1, \ldots, v_n$.
3. Initialise the DP at the source(s) and relax along the topological order.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1'000'000'007;

int main(){
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for(int i=0;i<m;i++){
        int u, v; cin >> u >> v; --u; --v;
        adj[u].push_back(v);
        indeg[v]++;
    }
    // Kahn's algorithm for topological order.
    queue<int> q;
    for(int i=0;i<n;i++) if(indeg[i] == 0) q.push(i);
    vector<int> topo;
    while(!q.empty()){
        int u = q.front(); q.pop();
        topo.push_back(u);
        for(int v : adj[u]) if(--indeg[v] == 0) q.push(v);
    }
    // dp[u] = number of paths from a designated source (say vertex 0) to u
    vector<ll> dp(n, 0);
```

```
    dp[0] = 1;
    for(int u : topo){
        for(int v : adj[u])
            dp[v] = (dp[v] + dp[u]) % MOD;
    }
    cout << dp[n-1] << "\n";
    return 0;
}
```

Tip 7.1: Counting vs optimising

The same skeleton handles counting (with + as the merge) and optimisation (with min or max). The topological order is the only thing that matters — it guarantees that when we process vertex u, all its predecessors have already been processed and we can read their finalised DP values.

7.3 Grid path counting

7.3.1 Canonical problem

Count the number of paths from $(1, 1)$ to (n, m) moving only right or down. This is the simplest DAG DP: each cell (i, j) has at most two predecessors $(i-1, j)$ and $(i, j-1)$, and the recurrence is

$$dp[i][j] = dp[i-1][j] + dp[i][j-1],$$

with $dp[1][1] = 1$.

```
int n, m; cin >> n >> m;
vector<vector<ll>> dp(n+1, vector<ll>(m+1, 0));
dp[1][1] = 1;
for(int i=1;i<=n;i++)
    for(int j=1;j<=m;j++){
        if(i == 1 && j == 1) continue;
        dp[i][j] = (dp[i-1][j] + dp[i][j-1]) % MOD;
    }
cout << dp[n][m] << "\n";
```

7.3.2 Variant: blocked cells

If some cells are blocked, simply initialise their dp to 0. The same recurrence handles obstacles without modification.

7.3.3 Variant: paths with modulo constraint

Count paths whose sum is divisible by k: augment the state with the current sum modulo k.

7.4 Longest path in a DAG

7.4.1 Canonical problem

Find the longest path (in edges) in a DAG. $O(n + m)$ via topological sort and a max-relaxation:

```
vector<int> dp(n, 0);
for(int u : topo){
    for(int v : adj[u]) dp[v] = max(dp[v], dp[u] + 1);
}
int ans = *max_element(dp.begin(), dp.end());
```

This is the canonical AtCoder DP G problem.

7.5 DP on a DAG with edge weights

For shortest path on a DAG with arbitrary edge weights (including negatives), the same skeleton relaxes by min:

```
const ll INF = 1e18;
vector<ll> dist(n, INF);
dist[0] = 0;
for(int u : topo) if(dist[u] < INF)
    for(auto [v, w] : adj[u])
        dist[v] = min(dist[v], dist[u] + w);
```

Because the graph is a DAG, there are no negative cycles and this is faster than Bellman-Ford.

7.6 Sum over paths

7.6.1 Canonical problem

Given a DAG with vertex weights w_u, compute

$$\sum_{P \in \text{paths}} \sum_{u \in P} w_u.$$

That is, sum over all source-to-sink paths of the sum of vertex weights along the path.

7.6.2 Recurrence

The trick is to maintain *two* DP values per vertex:

- $cnt[u]$ = number of paths from source to u.
- $sum[u]$ = sum over all source-to-u paths of the sum of weights on the path.

When we relax $u \to v$:

$$cnt[v] \mathrel{+}= cnt[u], \tag{7.1}$$

$$sum[v] \mathrel{+}= sum[u] + cnt[u] \cdot w_v. \tag{7.2}$$

The first equation copies $cnt[u]$ paths that now extend to v. The second equation copies $sum[u]$ (the existing path-sums), and adds w_v once for each new path, which is $cnt[u]$ times.

```
vector<ll> cnt(n, 0), sumw(n, 0);
cnt[0] = 1; sumw[0] = w[0];
for(int u : topo){
    for(int v : adj[u]){
        cnt[v]  = (cnt[v]  + cnt[u]) % MOD;
        sumw[v] = (sumw[v] + sumw[u] + cnt[u] * w[v]) % MOD;
    }
}
```

Tip 7.2: The dual-DP principle

Whenever the answer is a sum over paths, maintain *both* the count and the sum simultaneously. The two equations are linked; you cannot compute one without the other.

7.7 Sum over subsets (SOS DP)

7.7.1 Canonical problem

Given $f(S)$ for all $S \subseteq \{0, \ldots, n-1\}$ (a function on subsets of an n-element ground set), compute

$$g(S) = \sum_{T \subseteq S} f(T).$$

2^n subsets, naive $O(3^n)$ (since $\sum_S |S| \cdot 2^{|S|} = 3^n$). The *SOS DP* (sum-over-subsets) reduces this to $O(n \cdot 2^n)$.

7.7.2 Recurrence

The clever observation is that we can compute $g(S)$ *bit by bit*. Let $g_i(S)$ = "sum of $f(T)$ over $T \subseteq S$ such that T and S differ *only* in the first i bits". Then:

- $g_0(S)$: S has its low 0 bits fixed; just $f(S)$.
- $g_i(S) = g_{i-1}(S) + [\text{bit } i \text{ of } S \text{ is set}] \cdot g_{i-1}(S \setminus \{i\})$.

The recurrence says: if bit i is not set in S, then T cannot have bit i set either, so $g_i(S) = g_{i-1}(S)$. If bit i is set in S, T can have bit i either set or not; if not, we sum $g_{i-1}(S \setminus \{i\})$, if yes, we sum $g_{i-1}(S)$ (the case where bit i of T is forced equal to bit i of S).

```
// f[mask] is given. Compute g[mask] = sum_{sub \subseteq mask} f[sub].
vector<ll> g = f;        // copy
for(int i = 0; i < n; i++)
    for(int mask = 0; mask < (1<<n); mask++)
        if(mask & (1<<i))
            g[mask] += g[mask ^ (1<<i)];
```

Tip 7.3: Why iterate bit-first?

The outer loop is over *bits* i, not over masks. This is crucial: within a single outer iteration we ensure that all sums use bit i *uniformly*, so the recurrence is applied consistently. The inner loop over masks can be in any order, because $g[mask]$ is only updated from $g[mask \oplus 2^i]$, which is computed in the same outer iteration (when bit i is set in both).

7.7.3 Variants

- **Superset sum** $g(S) = \sum_{T \supseteq S} f(T)$: invert the condition — iterate i, and if bit i is *not* set in S, add from $S \cup \{i\}$.
- **Count of subsets with property**: replace f by an indicator.
- **Subset convolution**: compute $h(S) = \sum_{T \subseteq S} f(T) \cdot g(S \setminus T)$ in $O(n^2 \cdot 2^n)$ using the "ranked" variant of SOS DP.

7.8 Problem set

P1. AtCoder DP Contest, Problem G (Longest Path). $n, m \leq 10^5$ DAG. *Solution:* longest-path code in Section 7.4.

P2. CSES, Course Schedule. $n, m \leq 10^5$ DAG. *Solution:* topological sort, then $dp[u]$ = max path ending at u, with parent tracking for reconstruction.

P3. CSES, Path Queries II / Distinct Routes. Count edge-distinct paths from source to sink in a DAG. *Solution:* $cnt[v] = \sum_{(u,v)} cnt[u]$, modulo.

P4. Codeforces 1202E, You Are Given Some Strings... Suffix automaton + DAG DP. See Chapter 12.

P5. Codeforces 449D, Jzzhu and Numbers. Given $n \leq 10^6$ integers each $< 2^{20}$, count subsets whose bitwise-AND is 0. *Hint:* SOS DP on $f(S)$ = number of integers that are supersets of S; then inclusion-exclusion.

P6. Codeforces 165E, Compatible Numbers. Given $n \leq 10^6$ integers $a_i < 2^{22}$, for each a_i find some a_j ($j \neq i$) such that $a_i \& a_j = 0$. *Hint:* SOS DP to compute, for each mask S, some integer $a_j \subseteq \overline{S}$.

P7. Codeforces 914F, Substrings in a String. For each query (t, l, r) count occurrences of t in $s[l..r]$. Bitset trick combined with SOS-style meet-in-the-middle.

P8. AtCoder ABC 142 E, Get Everything. Knapsack-with-bitmask: state is "set of unlocked treasures".

7.9 Common pitfalls

- **DAG assumption.** If the graph has a cycle, longest path is NP-hard. Detect cycles (e.g. Kahn's algorithm finishes processing all vertices iff the graph is a DAG).
- **Modulo overflow in counting.** Use `long long` for intermediate products; take `%MOD` after every addition.
- **Wrong iteration direction in SOS DP.** The outer loop is over *bits*, not over masks. This is the only correct way.
- **Off-by-one in DAG source.** Make sure the source(s) are properly initialised (e.g. $cnt[s] = 1$, $dist[s] = 0$) and that no other vertex is initialised to a non-zero value.

Chapter 8

Bitmask DP

8.1 When bitmask DP is the right tool

A *bitmask DP* encodes a subset $S \subseteq \{0, 1, \ldots, n-1\}$ as an n-bit integer (so $0 \le S < 2^n$) and uses this integer as one component of the state. The pattern is appropriate whenever:

- $n \le 20$ (so $2^n \le 10^6$), and
- the problem asks for something defined over subsets — "order in which to visit all", "partition into pairs", "cover all vertices with minimum cost", etc.

The canonical examples are the Travelling Salesman Problem (TSP), Hamiltonian path / cycle, minimum-weight vertex cover, set cover (as decision), assignment problem, and partition-into-pairs problems.

8.2 The Travelling Salesman Problem (TSP)

8.2.1 Canonical problem

Given n cities and a complete directed graph with edge weights c_{ij}, find a tour that visits every city exactly once and returns to the start, minimising total cost. $n \le 18$ typical.

8.2.2 State

$dp[S][v]$ = "minimum cost to start at city 0, visit exactly the cities in the set S (with $|S| \ge 1$ and $0 \in S$), and end at city $v \in S$". Equivalently, we track which cities we have already visited and where we currently are.

8.2.3 Transition

If the current set is S and we are at $v \in S$, the previous step was "come to v from some $u \in S \setminus \{v\}$". Hence:

$$\boxed{dp[S][v] = \min_{u \in S \setminus \{v\}} \left(dp[S \setminus \{v\}][u] + c_{uv} \right)} \tag{8.1}$$

Base: $dp[\{0\}][0] = 0$ (started at 0, only 0 visited). Answer: $\min_v dp[\text{all}][v] + c_{v,0}$ (close the loop).

8.2.4 Iterating over subsets in the correct order

Tip 8.1: The cardinality-first order

Iterate masks in increasing order of *population count* (number of bits set). This guarantees that when computing $dp[S][v]$, the smaller set $S \setminus \{v\}$ has already been processed.

A simpler trick that always works: iterate masks in increasing numerical order. Since $S \setminus \{v\}$ has fewer bits than S, its numerical value is smaller, and a forward iteration from 0 to $2^n - 1$ suffices.

8.2.5 C++17 implementation

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main(){
    int n; cin >> n;
    vector<vector<ll>> c(n, vector<ll>(n));
    for(int i=0;i<n;i++) for(int j=0;j<n;j++) cin >> c[i][j];
    const ll INF = 1e18;
    int FULL = (1<<n) - 1;
    vector<vector<ll>> dp(1<<n, vector<ll>(n, INF));
    dp[1][0] = 0;                       // start at 0, set {0} visited
    for(int S = 1; S <= FULL; S++){
        for(int v = 0; v < n; v++) if(S & (1<<v)){
            for(int u = 0; u < n; u++) if((S & (1<<u)) && u != v){
                ll prev = dp[S ^ (1<<v)][u];
                if(prev < INF)
                    dp[S][v] = min(dp[S][v], prev + c[u][v]);
            }
        }
    }
    ll ans = INF;
    for(int v = 1; v < n; v++) if(dp[FULL][v] < INF)
        ans = min(ans, dp[FULL][v] + c[v][0]);
    cout << (ans == INF ? -1 : ans) << "\n";
    return 0;
}
```

8.2.6 Complexity

States: $2^n \cdot n$. Transitions per state: n. Total: $O(n^2 \cdot 2^n)$ time and $O(n \cdot 2^n)$ memory. For $n = 18$, this is $\approx 8.5 \cdot 10^7$ operations — comfortably within time, ~ 32 MB of memory.

Tip 8.2: Memory reduction

If you do not need to reconstruct the path, store only the current row by cardinality. Otherwise, store $parent[S][v] =$ the chosen u in the minimisation; reconstruct by repeatedly removing v.

8.3 Hamiltonian path / cycle

8.3.1 Canonical problem

Given an *arbitrary* directed graph (not necessarily complete), decide whether it contains a Hamiltonian path (visit each vertex once).

8.3.2 Recurrence

Same skeleton as TSP, but with 0/1 feasibility instead of min cost:

$$dp[S][v] = \bigvee_{u \in S \setminus \{v\},\ (u,v) \in E} dp[S \setminus \{v\}][u].$$

Answer: $\bigvee_v dp[\text{all}][v] = 1$ iff a Hamiltonian path exists.

To find a Hamiltonian *cycle*, additionally check the edge $(v, \text{start}) \in E$ in the final step.

8.3.3 Counting Hamiltonian paths

Replace the `OR` with `+` modulo $10^9 + 7$. Same skeleton.

8.3.4 Bitmask DP on edge constraints

A common sub-pattern: "find the shortest path from s to t that passes through each of the vertices in a given set $P \subseteq V$ at least once". For $|P| \leq 15$, run a TSP-style DP on P where the edge weights are the all-pairs shortest paths from s to t; this gives $O(|P|^2 \cdot 2^{|P|} + n^3)$ (Floyd-Warshall for the all-pairs).

8.4 Assignment problem

8.4.1 Canonical problem

Given an $n \times n$ cost matrix c_{ij}, find a permutation π minimising $\sum_i c_{i,\pi(i)}$.

8.4.2 Bitmask DP version

$dp[S]$ = "minimum cost to assign the first $|S|$ workers to the set S of jobs". Transition: for each unassigned job $j \notin S$, $dp[S \cup \{j\}] = \min dp[S \cup \{j\}], dp[S] + c_{|S|,j}$. Answer: $dp[\text{all}]$.

$O(n \cdot 2^n)$ time. For $n \leq 20$ this is $2 \cdot 10^7$, comfortably in time. The Hungarian algorithm solves the problem in $O(n^3)$ and is preferred for $n > 20$.

```
const ll INF = 1e18;
int n; cin >> n;
vector<vector<ll>> c(n, vector<ll>(n));
for(int i=0;i<n;i++) for(int j=0;j<n;j++) cin >> c[i][j];
vector<ll> dp(1<<n, INF);
dp[0] = 0;
for(int S = 0; S < (1<<n); S++){
    int i = __builtin_popcount(S);       // worker index = number of jobs taken
    for(int j = 0; j < n; j++) if(!(S & (1<<j))){
        dp[S | (1<<j)] = min(dp[S | (1<<j)], dp[S] + c[i][j]);
    }
}
cout << dp[(1<<n) - 1] << "\n";
```

8.5 Minimum-weight vertex cover / Steiner tree

8.5.1 Steiner tree on graphs

Given a graph and a set T of *terminal* vertices, find a connected subgraph of minimum weight that contains all of T. For $|T| \leq 15$ the bitmask DP gives a polynomial approximation:

- Compute all-pairs shortest paths $d(u, v)$ via Dijkstra from every vertex $u \in T$.
- $dp[S][v]$ = "minimum weight of a tree containing the terminals in $S \subseteq T$ and the vertex v".
- Transition 1 (Steiner merge): for $S' \subset S$, $dp[S][v] = \min_{S' \subset S} dp[S'][v] + dp[S \setminus S'][v]$.
- Transition 2 (Dijkstra): $dp[S][v] = \min_u dp[S][u] + d(u, v)$.

$O(3^{|T|} \cdot n + 2^{|T|} \cdot (n \log n + m))$.

8.6 Partition-into-pairs

8.6.1 Canonical problem

Given $2n$ objects and a cost c_{ij} for pairing objects i and j, find a perfect matching of minimum total cost. This is solvable with Hungarian in $O(n^3)$ but with bitmask DP it is $O(n \cdot 2^{2n})$ when $n \leq 13$ or so:

```
// c[i][j] = cost of pairing i and j (i < j)
vector<ll> dp(1<<(2*n), INF);
dp[0] = 0;
for(int S = 0; S < (1<<(2*n)); S++){
    // Pick the lowest unset bit, pair it with every other unset bit.
    int i = __builtin_ctz(~S & ((1<<(2*n)) - 1));
    if(i == 2*n) continue;       // S is full
    for(int j = i+1; j < 2*n; j++) if(!(S & (1<<j))){
        int S2 = S | (1<<i) | (1<<j);
        dp[S2] = min(dp[S2], dp[S] + c[i][j]);
    }
}
```

The trick of "always pair the lowest unset bit" avoids double-counting the order in which pairs are made and reduces the work by a factor of $2^n \cdot n!$, which is huge.

8.7 Iterating over submasks efficiently

```
// Enumerate all submasks of a given mask `m`, in decreasing order.
for(int sub = m; sub; sub = (sub - 1) & m){
    // do something with `sub`
}
// `sub == 0` is the last submask, handled outside the loop if needed.
```

This idiom iterates over all submasks of m in $O(2^{|m|})$ total time across the whole loop. It is the basis for SOS DP, subset convolution, and many "iterate over subsets" problems.

8.8 Problem set

P1. CSES, Hamiltonian Path. $n \leq 20$, $m \leq n^2$. *Solution:* the Hamiltonian-path code above.

P2. CSES, Bit Problem (Counting Coprime). $n \leq 20$. Bitmask enumeration.

P3. AtCoder ABC 142 E, Get Everything. $n \leq 1000$ keys, each opening some subset of treasures (bitmask). Find the minimum cost to open all. *Hint:* SOS DP over keys; the state is the set of treasures opened.

P4. Codeforces 1316E, Team Building. $n \leq 10^5$ players, $p \leq 7$ positions. Pick p starters and k substitutes. *Hint:* bitmask DP on the p positions; sort players by total value.

P5. Codeforces 1341F, Nastya and Time Machine. Path bitmask on a graph.

P6. Codeforces 1187C, Vasya and Array. $n \leq 5000$ subarray constraints. Solve via bitmask DP on prefix.

P7. Codeforces 407C, Triangle. $n \leq 10^6$. Use Lucas theorem + bitmask DP.

P8. Codeforces 1239D, Cat Party. $n \leq 10^5$. Use bitmask for groups, with linear DP.

P9. CSES, Counting Tilings. $n, m \leq 10$, count domino tilings of an $n \times m$ grid. *Hint:* bitmask DP over columns. $dp[col][mask]$ = "number of ways to tile the first *col* columns such that the right-most column's interaction with the next column is given by the bitmask *mask*".

8.9 Common pitfalls

- **Wrong iteration order in TSP.** Forward mask iteration works for TSP because $S \setminus \{v\}$ has fewer bits and thus smaller numerical value. For some variants (e.g. when a transition can add multiple bits) you must explicitly sort by population count.
- **Memory for $n = 22$.** $2^{22} \times 22 \times 8$ bytes ≈ 740 MB. Too large. Reduce to $O(2^n)$ by storing only the previous cardinality layer, if reconstruction is not needed.
- **Bit iteration idiom.** `(sub - 1) & m` is the correct idiom for "previous submask". Forgetting the `& m` gives wrong results.
- **Using `long long` for sets with $n > 30$.** If $n > 30$ you exceed 64 bits and need `bitset` or `array` of integers; reconsider whether bitmask DP is appropriate.

Chapter 9

Digit DP

9.1 The unifying idea

A *digit DP* is one whose state is a position in the decimal (or binary) expansion of a number, plus auxiliary flags. The pattern is appropriate whenever the problem asks "count integers in $[L, R]$ with property P" and $|R| \leq 10^{18}$. Because the value is too large to iterate over, we instead iterate over its digits — at most 19 of them — and use a small DP on those positions.

The key trick is the *tight flag*, which tells us whether the prefix we have built so far exactly matches the upper bound R. If yes, the next digit cannot exceed the corresponding digit of R; if no, the next digit can be anything in $[0, 9]$.

9.2 Canonical problem

Count integers in $[0, X]$ whose decimal digits satisfy some property (e.g. sum of digits equals S, contains no two consecutive equal digits, contains no digit 9, etc.).

9.3 State and transition

State. A tuple

$$(\text{position } i, \quad \text{tight flag}, \quad \text{other parameters}).$$

Here:

- i ranges over the digit positions, $0 \leq i < \text{len}$.
- tight $\in \{0, 1\}$ says whether we are still bounded by R at position i.
- Other parameters track whatever the property needs: the running digit-sum, the last digit (for adjacent-digit rules), the number of occurrences of some digit, etc.

Transition. At position i, choose a digit $d \in [0, 9]$ (or a smaller upper bound if tight $= 1$ and d would exceed the i-th digit of R). The new tight flag is

$$\text{tight}' = \text{tight} \wedge (d = R_i),$$

i.e. tight stays 1 only if we were already tight and we chose the maximal allowed digit.

Answer. The number of integers in $[0, X]$ with property P is the sum over all full-length states $(\text{len}, \text{tight}, \text{final})$ where the property is satisfied. To count $[L, R]$, compute $f(R) - f(L - 1)$.

9.4 A first worked example: sum of digits equals S

9.4.1 Problem

Count integers in $[0, X]$ whose sum of decimal digits equals S. $|X| \le 10^{18}$, $S \le 200$.

9.4.2 State

$dp[i][\text{tight}][s]$ = "number of ways to fill positions $i..len-1$ such that the resulting prefix is $\le R$ (taking the tight flag into account) and the sum of digits from i to the end is exactly s".

9.4.3 C++17 implementation

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

string num;            // decimal digits of X (most significant first)
int len;
ll S;

// dp[i][tight][s] : number of ways to fill positions i..len-1
//                   given current tight flag and remaining digit-sum s.
ll memo[20][2][200];

ll rec(int i, int tight, int s){
    if(i == len) return (s == 0) ? 1 : 0;
    if(s < 0) return 0;
    ll &res = memo[i][tight][s];
    if(res != -1) return res;
    res = 0;
    int upper = tight ? (num[i] - '0') : 9;
    for(int d = 0; d <= upper; d++){
        int ntight = tight && (d == upper);
        res += rec(i+1, ntight, s - d);
    }
    return res;
}

ll count_up_to(ll X){
    if(X < 0) return 0;
    num = to_string(X);
    len = num.size();
    memset(memo, -1, sizeof(memo));
    return rec(0, 1, S);
}

int main(){
    ll L, R; cin >> L >> R >> S;
    cout << count_up_to(R) - count_up_to(L-1) << "\n";
    return 0;
}
```

Tip 9.1: State space

The number of states is small: $\text{len} \times 2 \times (S+1) \approx 20 \times 2 \times 200 = 8000$ in the example. Each state has at most 10 transitions. Total work is $O(\text{len} \cdot S \cdot 10)$, i.e. $O(40{,}000)$ operations. This is why digit DP is so powerful: the "huge range" becomes a tiny state space.

9.5 Variants and the parameter zoo

The flexibility of digit DP comes from the "other parameters" axis. Some common parameters:

- **Running digit sum** (as above).
- **Last digit** (for rules like "no two adjacent digits are equal" or "no digit 0 after a non-zero digit").
- **Count of digit occurrences** ("how many 4s appear in $[0, X]$ total?").
- **Modulo** ("sum divisible by k"); state is $s \bmod k$.
- **Leading-zero flag** (digits 0 at the start of the number don't count; e.g. for the digit sum of a number, the digit 0 of the most significant position should not add to the sum unless the number is actually 0).
- **Position of last non-zero** (for problems like "X's decimal representation has at most k non-zero digits").

9.6 Iterative (non-recursive) digit DP

The recursive version above is cleaner. The iterative version removes the recursion overhead, which can matter when many queries are run on the same upper bound, but is more error-prone:

```
// Iterative digit DP. dp[i][tight][s] = number of ways.
vector<vector<vector<ll>>> dp(len+1, vector<vector<ll>>(2, vector<ll>(S+1, 0)));
dp[0][1][0] = 1;                  // empty prefix, tight, sum 0
for(int i=0;i<len;i++){
    for(int tight=0;tight<2;tight++){
        for(int s=0;s<=S;s++){
            if(!dp[i][tight][s]) continue;
            int upper = tight ? (num[i]-'0') : 9;
            for(int d=0; d<=upper; d++){
                int ntight = tight && (d==upper);
                int ns = s + d;
                if(ns > S) continue;
                dp[i+1][ntight][ns] += dp[i][tight][s];
            }
        }
    }
}
ll ans = dp[len][0][S] + dp[len][1][S];
```

9.7 Two-bound digit DP

When both L and R are bounds (not just $[0, R]$), there are two approaches:

1. **Subtraction trick.** Compute $f(R) - f(L-1)$ where f is a one-sided digit DP. This is what we did above.
2. **Direct two-bound DP.** Track two tight flags $(\text{tight}_L, \text{tight}_R)$. Slightly more code, but eliminates the subtraction — useful when the property is not a counting function (e.g. "sum over $[L, R]$ of the digit sum").

9.7.1 Counting the sum of digits over a range

For "find the sum of the decimal digit-sums of all $x \in [L, R]$", use a digit DP that returns a *pair* (cnt, sum) at each state: the number of integers and the sum of their digit-sums. The merge operator combines these pair-wise:

```cpp
// pair.first  = number of integers
// pair.second = sum of their digit-sums
pair<ll,ll> rec(int i, int tight){
    if(i == len) return {1, 0};
    auto &res = memo[i][tight];
    if(res.first != -1) return res;
    res = {0, 0};
    int upper = tight ? (num[i] - '0') : 9;
    for(int d = 0; d <= upper; d++){
        auto [c, s] = rec(i+1, tight && (d == upper));
        res.first  += c;
        res.second += s + c * d;        // every integer has digit d at position i
    }
    return res;
}
```

9.8 Automaton-based digit DP

If the property is "does not contain the substring T" (for some forbidden string T), the natural state is the position in the KMP-prefix automaton of T (i.e. the length of the longest prefix of T that is a suffix of the prefix we have built so far). The transition is the KMP failure-function transition on the chosen digit / character. This combines digit DP with the automaton DP of Chapter 12.

9.8.1 Canonical example

Count integers in $[1, X]$ whose decimal representation does not contain the substring "13".

9.8.2 State

$dp[i][\text{tight}][k]$ where k is the length of the longest prefix of "13" that is a suffix of the prefix built so far. The KMP automaton has $|T| + 1 = 3$ states (0, 1 for having seen "1" as the last char, 2 for the forbidden "13"). When $k = 2$ we are forbidden; otherwise we transition using the KMP failure function on the chosen digit.

```cpp
// 'forbidden' is the target substring.
// Build KMP failure array and the automaton transition function.
string fbd = "13";
int m = fbd.size();
vector<vector<int>> nxt(10, vector<int>(m+1));
for(int c=0;c<10;c++){
    for(int k=0;k<=m;k++){
        string s;
        for(int i=0;i<k;i++) s += fbd[i];
        s += char('0' + c);
        // Find longest proper prefix of fbd that is a suffix of s.
        int nk = 0;
        for(int len = min((int)s.size(), m); len >= 1; len--){
            if(s.substr(s.size()-len) == fbd.substr(0, len)){ nk = len; break; }
        }
        nxt[c][k] = nk;
    }
}
// Now in the digit DP, the state is (i, tight, k).
// If nxt[d][k] == m, the digit d takes us into the forbidden state -- skip.
```

9.9 Problem set

P1. SPOJ Cubes / NDS. Count integers up to X with digit-sum S.

P2. Codeforces 628D, Magic Numbers. $n \le 10^{18}$, count integers in $[L, R]$ divisible by m and whose digits are all from a subset of $\{0, \dots, 9\}$. *Hint:* digit DP with state $(i, \text{tight}, \text{mod}, \text{last})$.

P3. Codeforces 1076E, Digits. Count integers whose digit sum is divisible by k.

P4. AtCoder ABC 208 D, Digit Sum. $X \le 10^{18}$, $S \le 9 \cdot 18 = 162$. Count integers with digit sum S.

P5. Codeforces 1341D, Nastya and Time Machine. Hybrid with constructive DP.

P6. Codeforces 1245F, Binary String Reconstruction. Two-bound digit DP on binary representations.

P7. Codeforces 55D, Beautiful Numbers. Count integers divisible by the sum of their digits, modulo. The state is the LCM of possible digit sums ($= \text{LCM}(1, \dots, 9) = 2520$), giving $18 \times 2 \times 2520 \times 50$ states.

P8. Leetcode 902, Numbers At Most N Given Digit Set. $N \le 10^9$ and a set of allowed digits. *Solution:* digit DP with a tight flag and a per-position count of valid choices.

P9. SPOJ RANGESUM. Sum of digits over a range.

9.10 Common pitfalls

- **Leading zero handling.** The number 007 is just 7; its digit sum is 7, not 14. Use a `started` flag (or skip leading-zero digits specially).
- **Memoisation key.** The state tuple must include the tight flag; otherwise two different prefixes share a memo cell and give wrong counts.
- **Index direction.** Process digits most-significant first. The order matters because the tight flag depends on the prefix.
- $L = 0$ **edge case.** If $L = 0$ then $L - 1 = -1$ is invalid; handle it explicitly.
- **Modulo.** If the answer is huge, apply `%MOD` at every addition. If S is also bounded by 200, you can use `int` for the DP values; otherwise use `long long`.

Chapter 10

Game DP and Sprague–Grundy

10.1 The unifying idea

A *game DP* is one whose state is a game position and whose transitions are valid moves. The two fundamental questions:

- Who wins under optimal play? (*Decision version.*)
- What is the Grundy (a.k.a. Sprague–Grundy, nimber) value of the position, which lets us reason about sums of independent games?

For *impartial* games (where the set of legal moves from a position is the same for both players), the Sprague–Grundy theorem reduces the analysis to a single integer per position, and the **XOR** of these integers tells us who wins the disjunctive sum. This is the single most powerful tool in combinatorial game theory for contests.

10.2 Definitions

Definition 10.1: Impartial game

An *impartial game* is a position together with a set of legal moves, where the moves available from a position depend only on the position itself (not on whose turn it is). A position with no legal moves is a *terminal* (losing) position.

Definition 10.2: Grundy value

The *Grundy value* (or *Sprague–Grundy value*, or *nimber*) of a game position G, written $g(G)$, is defined recursively:

$$g(G) = \operatorname{mex}\{\, g(G') : G \to G' \text{ is a legal move} \,\}, \tag{10.1}$$

where mex of a set of non-negative integers is the smallest non-negative integer *not* in the set.

Theorem 10.1: Sprague–Grundy theorem

Every impartial game under the normal-play convention (the player who cannot move loses) is equivalent to a single Nim heap of size $g(G)$. In particular, a disjunctive sum $G_1 + G_2 + \cdots + G_k$ of impartial games is a P-position (previous player wins, i.e. current player loses) iff

$$g(G_1) \oplus g(G_2) \oplus \cdots \oplus g(G_k) = 0, \tag{10.2}$$

where $\oplus$ is bitwise XOR.

Tip 10.1: What "equivalent to a Nim heap" means

A Nim heap of size g is a position where the current player can move to heaps of sizes $0, 1, \ldots, g-1$, and these are the only moves. The mex definition makes $g(G)$ the unique integer such that the moves of G *include* all Grundy values $0, 1, \ldots, g(G)-1$, and *no* move has Grundy value $g(G)$ itself. This is precisely the Nim-heap property.

10.3 Canonical example: Nim

10.3.1 Problem

There are n piles of stones. On a turn, a player chooses a non-empty pile and removes any positive number of stones. The player who cannot move (all piles empty) loses. Decide who wins.

10.3.2 Analysis

The Grundy value of a pile of k stones is exactly k, because the moves are "reduce to $0, 1, \ldots, k-1$" and the mex of $\{0, 1, \ldots, k-1\}$ is k. By the Sprague–Grundy theorem, the sum of n piles is a P-position iff the XOR of pile sizes is 0.

```
ll x = 0;
for(int i=0;i<n;i++) x ^= a[i];
cout << (x ? "First\n" : "Second\n");
```

10.4 Computing Grundy values

For an arbitrary impartial game on a finite DAG of positions, the Grundy values are computed by reverse topological sort:

```
// adj[u] = list of legal moves from u (as edge u -> v).
// Compute topological order first (terminal positions have out-degree 0).
vector<int> topo = topological_sort(adj);
vector<int> g(n, 0);
for(int u : reverse(topo)){
    set<int> reachable;
    for(int v : adj[u]) reachable.insert(g[v]);
    int mex = 0;
    while(reachable.count(mex)) mex++;
    g[u] = mex;
}
```

The cost is $O(n+m)$ for the topological sort plus $O(\sum \text{out-degrees})$ for the mex computation. For $n \le 10^5$ this is fast; for $n \le 10^6$ consider bitset tricks for the mex.

10.5 Patterns of Grundy values

Many games have a Grundy value that is periodic in the position size. Knowing the period saves the DP and gives an $O(1)$ answer:

- **Nim.** $g(k) = k$.
- **Subtraction games.** Allowed moves: remove any of $\{a_1, \ldots, a_t\}$. The Grundy sequence is eventually periodic; the period is at most $2^{\max a_i}$ (so finding it by DP is fast).
- **Wythoff Nim.** Two piles, moves: remove from one pile or remove the same number from both. The P-positions are $(\lfloor k\varphi \rfloor, \lfloor k\varphi^2 \rfloor)$ for $k = 0, 1, \ldots$ where φ is the golden ratio.

- **Green Hackenbush on a tree.** The Grundy value of a tree is the XOR of the (Grundy value +1) of each child. This generalises to many "cut the leaves" games.

10.5.1 Green Hackenbush on a tree

```cpp
// adj is a rooted tree; root = 0.
function<int(int,int)> dfs = [&](int u, int p) -> int {
    int g = 0;
    for(int v : adj[u]) if(v != p)
        g ^= (dfs(v, u) + 1);
    return g;
};
int grundy = dfs(0, -1);
// First player wins iff grundy != 0.
```

The "+1" comes from the fact that cutting an edge leaves a subtree whose Grundy value is one less (in the Nim-heap analogy).

10.6 Composite games and the XOR trick

10.6.1 Pattern recognition

A composite game is a sum of independent sub-games; the XOR-trick applies whenever:

- The game is the disjunctive sum of k *independent* sub-games (a move chooses one sub-game and moves in it).
- Each sub-game is impartial (same move-set for both players).
- Normal-play convention (last mover wins).

10.6.2 Canonical composite example

n piles of stones. Each pile is an independent Nim game. The Grundy value of the composite is the XOR of the per-pile values.

10.6.3 Beyond Nim: composite of arbitrary games

A composite game may have several different types of sub-games. The Sprague–Grundy theorem still applies: compute the Grundy value of each sub-game independently, then XOR them.

10.7 Misère play

In *misère play*, the player who cannot move *wins*. The Sprague–Grundy theorem does not directly apply. However, for Nim specifically, the misère variant has a simple analysis:

- If all piles have size ≤ 1, the winning condition is the opposite of normal play: the first player wins iff the number of non-empty piles is even.
- Otherwise, the normal-play XOR rule still applies.

For general misère games, the analysis is far more complex (the "misère quotient" theory of Plambeck and Siegel).

10.8 Partisan games

In a *partisan* game, the move-set depends on whose turn it is (chess, checkers, etc.). The Sprague–Grundy theorem does not apply; the right tool is *Conway's surreal numbers* (a.k.a. combinatorial

game theory). These are rare in contests and beyond the scope of this book; the canonical reference is *Winning Ways* by Berlekamp, Conway, and Guy.

10.9 Problem set

P1. CSES, Nim Game. $n \le 10^5$ piles. *Solution:* XOR of pile sizes.

P2. Codeforces 1382B, Nim Game with Single Move. Sequential Nim where each player removes from the leftmost non-empty pile only. *Hint:* the first player wins iff the first non-empty pile has size > 1, because the first player can take all of it (or leave exactly one to force the opponent).

P3. Codeforces 1451C, String Equality. Hybrid with counting; not strictly a game.

P4. SPOJ GAME (Wythoff Nim). The P-positions are $(\lfloor k\varphi \rfloor, \lfloor k\varphi^2 \rfloor)$; compute them iteratively for k up to $\sqrt{\max n}$.

P5. Codeforces 1452C, Next. Composite impartial game.

P6. Codeforces 1382A, Common Subsequence. Game DP with composite structure.

P7. Codeforces 1265E, Beautiful Mirrors. Probability DP combined with game.

P8. CSES, Grid Paths / Staircase Nim. Staircase Nim is a variant of Nim where stones on odd-indexed piles behave like normal Nim, and stones on even-indexed piles are irrelevant. The Grundy value is the XOR of odd-indexed piles.

P9. Codeforces 1221E, Game With Stones. Staircase Nim on a 1-D row.

10.10 Common pitfalls

- **Missing the impartiality condition.** The Sprague–Grundy theorem *requires* that both players have the same move sets. If the game has different moves for the two players (partisan), it does not apply.
- **Misère vs normal.** Confirm which play convention the problem uses. Most contest games use normal play; misère requires different analysis.
- **Composite game independence.** The XOR trick requires *independent* sub-games — a move must touch exactly one sub-game. If moves can span multiple sub-games, the analysis is more delicate.
- **Large state space.** If the number of game positions is huge, look for periodicity in the Grundy sequence (most subtraction games have period $\le 2^{\max a_i}$).

Chapter 11

Matrix Exponentiation

11.1 The unifying idea

Many DP problems have a *linear* recurrence of bounded degree. For example, the Fibonacci recurrence $F_n = F_{n-1} + F_{n-2}$ is linear in F_{n-1} and F_{n-2}. Such recurrences can be written as a matrix multiplication:

$$\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n-1} \\ F_{n-2} \end{pmatrix}.$$

Raising the matrix to the n-th power, then multiplying by the base vector, gives the answer in $O(k^3 \log n)$ time — exponentially better than the $O(n)$ DP. This is the right tool whenever:

- the recurrence is *linear* in the previous states,
- the degree k of the recurrence is small ($k \leq 50$),
- n is huge ($n \leq 10^{18}$).

11.2 Linear recurrences and matrix form

Definition 11.1: Linear recurrence

A sequence $a_0, a_1, a_2, \ldots$ satisfies a *linear recurrence of degree k with constant coefficients* if there exist constants $c_1, c_2, \ldots, c_k$ such that for all $n \geq k$,

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}.$$

11.2.1 Companion matrix

The recurrence can be expressed as

$$\begin{pmatrix} a_n \\ a_{n-1} \\ \vdots \\ a_{n-k+1} \end{pmatrix} = \underbrace{\begin{pmatrix} c_1 & c_2 & \cdots & c_{k-1} & c_k \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}}_{M} \begin{pmatrix} a_{n-1} \\ a_{n-2} \\ \vdots \\ a_{n-k} \end{pmatrix}. \tag{11.1}$$

The matrix M is called the *companion matrix* of the recurrence.

By induction:

$$\begin{pmatrix} a_n \\ a_{n-1} \\ \vdots \\ a_{n-k+1} \end{pmatrix} = M^{n-k+1} \begin{pmatrix} a_{k-1} \\ a_{k-2} \\ \vdots \\ a_0 \end{pmatrix}. \tag{11.2}$$

So computing a_n is the same as computing M^{n-k+1} (a $k \times k$ matrix raised to a power of size up to 10^{18}), then multiplying by the initial state vector. With fast exponentiation this is $O(k^3 \log n)$ time, $O(k^2)$ memory.

11.3 Fast matrix exponentiation

The standard *binary exponentiation* works for matrices just as for integers:

```cpp
using Matrix = vector<vector<ll>>;
const ll MOD = 1'000'000'007;

Matrix mul(const Matrix& A, const Matrix& B){
    int n = A.size(), m = B[0].size(), p = B.size();
    Matrix C(n, vector<ll>(m, 0));
    for(int i=0;i<n;i++) for(int k=0;k<p;k++){
        if(A[i][k]==0) continue;
        for(int j=0;j<m;j++)
            C[i][j] = (C[i][j] + A[i][k]*B[k][j]) % MOD;
    }
    return C;
}

Matrix power(Matrix A, ll e){
    int n = A.size();
    Matrix R(n, vector<ll>(n, 0));
    for(int i=0;i<n;i++) R[i][i] = 1;   // identity
    while(e > 0){
        if(e & 1) R = mul(R, A);
        A = mul(A, A);
        e >>= 1;
    }
    return R;
}
```

Tip 11.1: Strassen's not worth it

For contest-sized k ($k \leq 50$), naive $O(k^3)$ matrix multiplication is faster than Strassen's $O(k^{2.81})$ due to constants. Stick with the naive multiplication. Optimise by skipping 0-valued entries in sparse matrices (the companion matrix has only $2k - 1$ non-zero entries).

11.4 Canonical example: Fibonacci to $F_{10^{18}}$

```cpp
int main(){
    ll n; cin >> n;
    Matrix M = {{1,1},{1,0}};
    Matrix R = power(M, n);
    // F_n = R[1][0] = R[0][1] (either off-diagonal entry).
    cout << R[0][1] << "\n";
    return 0;
}
```

This computes F_n in $O(\log n)$ modulo $10^9 + 7$, even for n up to 10^{18}. The classical $O(n)$ DP would TLE for $n > 10^8$.

11.5 Tiling DP with matrix exponentiation

11.5.1 Canonical problem

Count the number of ways to tile a $2 \times n$ board with 2×1 dominos. Let a_n be this count. The recurrence $a_n = a_{n-1} + a_{n-2}$ is derived from the fact that a $2 \times n$ tiling either starts with two horizontal dominos (leaving a $2 \times (n-2)$ board) or with a single vertical domino (leaving a $2 \times (n-1)$ board). So $a_n = a_{n-1} + a_{n-2}$, $a_0 = 1$, $a_1 = 1$ — this is just the Fibonacci sequence shifted by one.

```
// Tiling of 2 x n with dominos, n up to 1e18.
Matrix M = {{1,1},{1,0}};   // (a_n, a_{n-1}) = M (a_{n-1}, a_{n-2})
Matrix R = power(M, n);     // we want a_n
// (a_n, a_{n-1}) = M^{n-1} (a_1, a_0) = M^{n-1} (1, 1).
// So a_n = R[0][0] + R[0][1] (with appropriate R = M^{n-1}).
```

11.5.2 Variants

- $3 \times n$ **tiling with** 2×1 **dominos.** Only possible when n is even; the recurrence has more state.
- $4 \times n$ **tiling.** Larger state; same matrix exponentiation.
- **Tiling with dominos and trominos.** State includes "gap" types.
- **Coloured tilings.** Multiply by the number of valid colourings.

11.6 Recurrence on a graph (number of walks)

11.6.1 Canonical problem

Given a (small) directed graph with $n \leq 50$ vertices, count the number of walks of length L from u to v, for $L \leq 10^{18}$.

11.6.2 Matrix formulation

Let A be the adjacency matrix of the graph. The number of walks of length L from u to v is $(A^L)_{u,v}$. So raise A to the L-th power with fast matrix exponentiation.

This is the cleanest application of matrix exponentiation to a DP-style problem: the "DP state" is the vertex, and the "transition" is the adjacency matrix.

11.7 Linear recurrences with non-constant terms

If the recurrence has the form

$$a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k} + f(n),$$

where $f(n)$ is a polynomial in n, augment the state vector with additional dimensions tracking powers of n and the constant terms. The resulting matrix is slightly larger but the technique is the same.

11.7.1 Example: $a_n = a_{n-1} + n$

Augment the state to $(a_n, n, 1)$. The transition is

$$\begin{pmatrix} a_n \\ n \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} a_{n-1} \\ n-1 \\ 1 \end{pmatrix}.$$

Then a_n is the first component of $M^{n-1}(a_0, 0, 1)^T$.

11.8 Cayley–Hamilton and Berlekamp–Massey

For a sequence satisfying an unknown linear recurrence, the *Berlekamp–Massey* algorithm finds the recurrence from $2k$ sequence values. Combined with matrix exponentiation, this gives a complete pipeline: "measure $2k$ values of the sequence, find the recurrence, exponentiate to compute a_n for huge n".

11.8.1 Berlekamp–Massey in C++

```cpp
// Returns the minimal recurrence coefficients c such that
//   a[i] = sum_{j=1..k} c[j] * a[i-j]   for all i >= k.
vector<ll> berlekamp_massey(vector<ll> a, ll mod){
    vector<ll> ls, cur;  int lf = 0; ll ld = 0;
    for(int i=0;i<(int)a.size();i++){
        ll t = 0;
        for(int j=0;j<(int)cur.size();j++) t = (t + cur[j]*a[i-1-j]) % mod;
        if((a[i] - t) % mod == 0) continue;
        if(cur.empty()){ cur.resize(i+1); lf = i; ld = (a[i]-t)%mod; continue; }
        ll k = (a[i] - t) % mod * modpow(ld, mod-2, mod) % mod;
        vector<ll> c(i - lf - 1, 0); c.push_back(k);
        for(int j=0;j<(int)ls.size();j++) c.push_back(-ls[j]*k % mod);
        if(c.size() < cur.size()) c.resize(cur.size());
        for(int j=0;j<(int)cur.size();j++) c[j] = (c[j] + cur[j]) % mod;
        if((int)c.size() >= (int)cur.size() && i - (int)c.size() >= lf){
            ls = cur; lf = i; ld = (a[i] - t) % mod;
        }
        cur = c;
    }
    for(auto &x : cur) x = (x % mod + mod) % mod;
    return cur;
}
```

Tip 11.2: Berlekamp–Massey in contests

The trick is to compute the first $2k$ values of a sequence by a slow DP, run Berlekamp–Massey to discover the recurrence, then compute a_n for huge n via Cayley–Hamilton (which is $O(k^2 \log n)$ rather than $O(k^3 \log n)$). The Cayley–Hamilton reduction is essential when $k \geq 100$.

11.9 Problem set

P1. CSES, Graph Paths I. $n, m \leq 100$, $L \leq 10^{18}$. Count paths of length L from u to v. *Solution:* matrix exponentiation on the adjacency matrix.

P2. CSES, Graph Paths II. Same, but *minimum* length-L walk. Edge weights are non-negative; the matrix is the all-pairs shortest paths and the "multiplication" is replaced by min-plus (tropical) multiplication.

P3. Codeforces 1182A, Filling Shapes. Tiling a $3 \times n$ grid. *Solution:* recurrence $a_n = 4a_{n-1} - a_{n-2}$; matrix exponentiation.

P4. Codeforces 621E, Wet Shark and Blocks. $n \le 10^5$ blocks each with a digit; count the number of sequences of length L whose value is divisible by k. *Solution:* matrix exponentiation on a $k \times k$ state of (current mod).

P5. Codeforces 506C, Mr. Kitayuta's Colorful Graph. Hybrid matrix exponentiation.

P6. Codeforces 1132E, Knapsack-on-a-Grid. Hybrid with knapsack; small DP + matrix exponentiation for the remaining steps.

P7. AtCoder ABC 129 F, Takahashi's Basics. Linear transformation matrix with fast exponentiation.

P8. SPOJ FIBOSUM. Sum of Fibonacci numbers $F_0 + \cdots + F_n$ modulo $10^9 + 7$. *Solution:* $\sum_{i=0}^{n} F_i = F_{n+2} - 1$. Use matrix exponentiation to compute F_{n+2}.

11.10 Common pitfalls

- **Modulo everywhere.** Multiplications can overflow 2^{63} when the modulus is close to $10^9 + 7$ and entries approach 10^{18}. Use `long long` and reduce after every multiplication.
- **Wrong matrix size.** The companion matrix for a degree-k recurrence is $k \times k$. The initial state vector has k entries $(a_{k-1}, \ldots, a_0)$.
- **Off-by-one in the exponent.** If $\vec{v}_n = M^{n-k+1}\vec{v}_{k-1}$, then a_n is the first component of the result. Test on a small example to verify the exponent.
- **Non-commutative matrices.** In the multiplication, $A \cdot B \neq B \cdot A$. Always maintain the multiplication order; $A^e = A^{e-1} \cdot A$ (right-multiply) or $A \cdot A^{e-1}$ (left-multiply) — they are equal for a single matrix, but in generalised problems with multiple matrices order matters.

Chapter 12

String and Automaton DP

12.1 The unifying idea

A *string DP* is one whose state is an index into a string (or pair of indices) together with auxiliary information about what has been matched so far. The auxiliary information is often captured as a *state of an automaton* (KMP, Aho-Corasick, suffix automaton). This converts string problems into graph-DP problems on the automaton vertices, where the edge labels correspond to appending characters and the DP state is "current automaton vertex + remaining budget".

The canonical examples:

- **Pattern match DP.** Number of strings of length n that do not contain a forbidden pattern.
- **Suffix automaton DP.** Number of distinct substrings; longest common substring; k-th lexicographic substring.
- **Automaton-based digit DP.** Count of integers in $[L, R]$ whose decimal representation does not contain any forbidden substring.

12.2 A first example: KMP automaton DP

12.2.1 Canonical problem

Given a forbidden string T of length m and a length n, count the number of strings of length n over alphabet Σ that do not contain T as a substring. $|T| \le 20$, $|\Sigma| \le 26$, $n \le 10^9$.

12.2.2 State

$dp[i][k]$ = number of length-i prefixes such that the longest prefix of T that is a suffix of the prefix has length k ($0 \le k \le m$). The transition: append any character $c \in \Sigma$; the new state k' is given by the KMP failure function on c from state k.

If $k' = m$, we have just matched T — so the new state is forbidden and we do not count this transition.

12.2.3 Recurrence

$$dp[i+1][k'] = \sum_{c \in \Sigma} \sum_{k \,:\, \mathrm{trans}(k,c)=k', k' \neq m} dp[i][k]. \tag{12.1}$$

12.2.4 Complexity and matrix exponentiation

$O(m)$ states, $O(|\Sigma|)$ transitions per state, $O(m|\Sigma|)$ per DP step. With n up to 10^9 we cannot iterate i up to n. The DP is linear: $dp_{i+1} = A\, dp_i$ where A is the $m \times m$ automaton transition matrix. Use matrix exponentiation (Chapter 11): $dp_n = A^n \cdot dp_0$ in $O(m^3 \log n)$.

```cpp
// Build the KMP automaton transition function.
vector<vector<int>> build_kmp(const string& pat){
    int m = pat.size();
    vector<int> fail(m+1, 0);
    for(int i=1;i<m;i++){
        int k = fail[i];
        while(k && pat[i]!=pat[k]) k = fail[k-1];
        fail[i+1] = (pat[i]==pat[k]) ? k+1 : 0;
    }
    // nxt[c][k] = new state after reading char c from state k.
    vector<vector<int>> nxt(26, vector<int>(m+1));
    for(int c=0;c<26;c++) for(int k=0;k<=m;k++){
        int nk = k;
        while(nk == m || (nk && pat[nk] != char('a'+c))) nk = fail[nk];
        nxt[c][k] = (pat[nk]==char('a'+c)) ? nk+1 : 0;
    }
    return nxt;
}
```

12.3 Aho-Corasick automaton DP

When multiple forbidden patterns are given, build the Aho-Corasick automaton. Each state is a node of the trie enriched with failure links. The DP state is (current automaton vertex, remaining length). The matrix-exponentiation trick applies just as in the KMP case; the matrix size is now the number of AC states (sum of pattern lengths).

12.3.1 Output links

For each state, store a flag "is this state the end of some forbidden pattern (or has such a state in its failure chain)?". If so, this state is "dangerous" and the DP excludes transitions leading into it.

```cpp
// After building the AC automaton (trie + failure links), propagate
// the "dangerous" flag along failure links: a state is dangerous
// if itself or any state in its failure chain is the end of a pattern.
for(int u : bfs_order)          // process failure links in BFS order
    if(fail[u] != u)
        dangerous[u] |= dangerous[fail[u]];
```

12.4 Suffix automaton DP

12.4.1 Canonical problem

Given a string s of length $n \leq 10^5$, count the number of distinct substrings of s. Equivalently, find the number of distinct paths in the suffix automaton starting from the initial state.

12.4.2 Suffix automaton facts

The suffix automaton of s is the minimal DFA accepting all suffixes of s. It has at most $2n-1$ states and $3n-4$ transitions. Every substring of s corresponds to exactly one path in the automaton starting at the initial state — but a path that ends in a state corresponds to a substring whose length is at most len(state) and more than len(link(state)).

12.4.3 Counting distinct substrings

The number of distinct substrings is

$$\sum_{v \neq \text{init}} (\text{len}(v) - \text{len}(\text{link}(v))).$$

Each state v contributes the substrings whose lengths are between $\text{len}(\text{link}(v)) + 1$ and $\text{len}(v)$, and these ranges partition the set of all substrings.

```
// After building SAM with states 0..sz-1 (state 0 is the initial).
long long distinct_substrings = 0;
for(int v = 1; v < sz; v++)
    distinct_substrings += len[v] - len[link[v]];
```

12.4.4 Counting occurrences of each substring

For each state v, the number of occurrences of any substring that ends at v equals the number of times v (or any state in its "link-tree" subtree) was visited during the SAM construction. This is computed with a DP on the link tree:

```
// Mark terminal states (states corresponding to suffixes of the whole string).
for(int v = last; v; v = link[v]) terminal[v] = 1;
// Topologically sort states by length (which is a topological order of the
// link tree), then propagate.
for(int v = 1; v < sz; v++) cnt[link[v]] += cnt[v];
```

12.5 Longest Common Substring of two strings

12.5.1 Canonical problem

Given s and t, find the length of their longest common substring (contiguous, not subsequence).

12.5.2 Solution via SAM

Build the SAM of s. Walk t through the automaton, maintaining a state v and a current length ℓ. For each character c of t:

- If v has a transition on c, follow it and increment ℓ.
- Else, follow suffix links until a state with a c-transition is found, or until $v =$ initial state. Update ℓ to $\text{len}(v)$ (the length of the longest substring ending at v).

Track the maximum ℓ over the walk.

```
int v = 0, l = 0, ans = 0;
for(char c : t){
    while(v && !trans[v].count(c)){ v = link[v]; l = len[v]; }
    if(trans[v].count(c)){ v = trans[v][c]; l++; }
    ans = max(ans, l);
}
```

This is $O(|s|+|t|)$ time and is dramatically faster than the $O(|s| \cdot |t|)$ DP for LCS-subsequence.

12.6 DP on suffix automaton: number of distinct substrings of length k

For each state v, the substrings that *end* at v have lengths in $(\text{len}(\text{link}(v)), \text{len}(v)]$. So the number of distinct substrings of length exactly k is the count of v's for which this range contains k:

```
vector<ll> cnt_k(n+1, 0);
for(int v = 1; v < sz; v++){
    int lo = len[link[v]] + 1, hi = len[v];
    if(lo <= hi){
        cnt_k[lo] += 1; cnt_k[hi+1] -= 1;   // difference array
    }
}
for(int i=1;i<=n;i++) cnt_k[i] += cnt_k[i-1];
```

12.7 Pattern match DP: edit distance with allowed operations

The LCS DP of Section 4.3 is the simplest pattern-match DP. Many variants exist:

- **Edit distance with weighted operations.** Each of insert, delete, substitute has its own cost; the DP simply uses the costs instead of 1.
- **Edit distance with at most k errors (approximate matching).** Use the Ukkonen algorithm $O(k \cdot \min(|s|, |t|))$ for small k; alternatively bit-parallel algorithms.
- **Wildcard matching.** The pattern can contain `?` (single character) and `*` (any sequence). State $dp[i][j]$ = "does pattern prefix i match text prefix j?".

12.8 Problem set

P1. Codeforces 1202E, You Are Given Some Strings... Given strings s and many t_i, count occurrences of any $t_i + t_j$ (i, j can be equal) as substrings of s. *Hint:* build Aho-Corasick of the t_i's and of the reversed t_i's; for each position p in s, compute (number of t_i ending at p)$\times$ (number of reversed t_j ending at $|s| - p - 1$).

P2. Codeforces 1238E, Keyboard Purchase. $n \leq 20$ letters in keyboard; cost is the number of fingers crossing the keyboard. *Hint:* bitmask DP on letters already placed.

P3. SPOJ SUBST1, New Distinct Substrings. Count distinct substrings of a string of length up to $5 \cdot 10^4$. *Solution:* SAM, sum of len[v] − len[link[v]].

P4. SPOJ NSUBSTR. For each $k = 1..n$, count the number of substrings of s that appear exactly k times. *Hint:* SAM with occurrence count DP (Section 12.4).

P5. AtCoder ABC 206 F, Interval Game. Hybrid with interval DP.

P6. Codeforces 432D, Prefixes and Suffixes. For each prefix $s[1..k]$, find the largest $l < k$ such that $s[1..l]$ is both a prefix and a suffix of $s[1..k]$, and count occurrences. *Hint:* KMP automaton with DP.

P7. Codeforces 1239D, Cat Party. Suffix automaton DP combined with rerooting.

P8. Codeforces 700B, Connecting Universities. Tree DP combined with suffix-automaton counting.

12.9 Common pitfalls

- **Failure link vs suffix link.** KMP's *failure function* and SAM's *suffix link* are different objects; the KMP failure is "the longest proper prefix of the pattern that is a suffix of the current prefix", while the SAM suffix link is the state corresponding to the longest suffix that occurs elsewhere in s. Mixing the two terminologies is a frequent source of bugs.
- **Memory for SAM on large strings.** For $|s| = 10^6$, the SAM has up to $2 \cdot 10^6$ states and up to $3 \cdot 10^6$ transitions. Use `unordered_map` for transitions or a pre-allocated array indexed by character.
- **Modulo in pattern DP.** Apply `%MOD` after every addition; the DP values can be huge.
- **Off-by-one in matrix exponentiation.** The exponent for "length-n strings" is n when dp_0 is the empty-prefix state. Verify on a small example.

Chapter 13

Probability and Expected Value DP

13.1 The unifying idea

A *probability DP* or *expected value DP* is one whose "value" at each state is a probability (a rational in $[0, 1]$) or an expected value (an arbitrary non-negative real). The transitions are the same as in deterministic DP, but the merge operation is *expectation-preserving* rather than *extremal*.

In competitive programming, probabilities and expectations are essentially always computed modulo $p = 10^9 + 7$ (or another large prime). To do this, we treat $1/p$ as $p - p \cdot \text{inverse}(p)$ ... more practically: replace every division by a multiplication with the modular inverse of the denominator, computed via Fermat's little theorem $a^{-1} \equiv a^{p-2} \pmod{p}$.

13.2 Modular arithmetic prerequisites

```
const ll MOD = 1'000'000'007;
ll modpow(ll a, ll e){ ll r=1; a%=MOD; while(e){ if(e&1) r=r*a%MOD;
    a=a*a%MOD; e>>=1; } return r; }
ll inv(ll a){ return modpow(a, MOD-2); }      // a^{-1} mod p
```

Every probability $p = a/b$ is represented as $a \cdot \text{inv}(b) \bmod \text{MOD}$.

13.3 Linearity of expectation

Theorem 13.1: Linearity of expectation

For any (possibly dependent!) random variables $X_1, X_2, \ldots, X_n$,

$$\mathbb{E}\left[\sum_i X_i\right] = \sum_i \mathbb{E}[X_i].$$

No independence is required.

Tip 13.1: Why this matters in DP

A large fraction of "expected value" problems become trivial once you spot that the value is a sum of indicator variables. "Expected number of distinct elements in a randomly permuted array" = sum over elements of Pr[this element appears somewhere $\times$ the indicator "this is the first occurrence of its value"]. Each expectation is a one-line computation.

13.4 Canonical example: expected number of coin flips

13.4.1 Problem

A coin has probability p of landing heads. We flip it until we get heads. What is the expected number of flips?

13.4.2 Two derivations

(a) Direct. Let X be the number of flips. Then $\Pr[X = k] = (1-p)^{k-1} \cdot p$, and

$$\mathbb{E}[X] = \sum_{k \geq 1} k \cdot (1-p)^{k-1} p = \frac{1}{p}.$$

(b) DP. Let E be the expected number of additional flips starting from "no heads yet". With one flip we either get heads (probability p, costing 1) or tails (probability $1-p$, costing 1 plus the expected number of flips from the same state):

$$E = p \cdot 1 + (1-p) \cdot (1 + E).$$

Solving: $E = 1/p$.

This is the prototypical "expected value DP" recurrence:

- The DP value is the expected value-to-go from the current state.
- The transition is a one-step expectation.
- Base cases are zero-expected-value terminals.

13.5 Random walks on graphs

13.5.1 Canonical problem

A particle starts at vertex s of a directed graph. At each step, it moves to a uniformly random out-neighbour. What is the expected number of steps to reach vertex t?

13.5.2 Recurrence

$E[u]$ = expected number of steps from u to t. For each $u \neq t$:

$$E[u] = 1 + \frac{1}{\deg(u)} \sum_{v \in N^+(u)} E[v].$$

$E[t] = 0$.

This is a *system of linear equations*, not a recursive DP in the usual sense (because u's equation depends on its successors, which may in turn depend on u if the graph has cycles). To solve:

- If the graph is a DAG, topologically sort and compute in reverse order — linear time.
- If the graph has cycles, solve the linear system by Gaussian elimination, $O(n^3)$. For $n \leq 500$ this fits.
- If the graph is special (e.g. tree), derive an explicit formula via tree DP (similar to rerooting).

13.5.3 Pseudo-polynomial time via "step cap"

If a probabilistic process is expected to terminate quickly (e.g. random walk on a path), a DP that bounds the number of steps can be practical: $E[u][k]$ = expected value from u with at most k steps remaining.

13.6 Expected value of "maximum element"

13.6.1 Problem

Given n numbers $a_1, \ldots, a_n$ and a random permutation, what is the expected maximum prefix-sum (over prefixes)?

13.6.2 Reduction

By linearity of expectation, the expected maximum is the sum over all prefix positions of the probability that this prefix achieves the maximum. Compute the probability by inclusion-exclusion; the DP is on the permutation space, often via bitmask.

13.7 Probability DP for "count of ways to reach a state"

Often the expected value is easier to compute by first counting the number of ways to reach each state, then dividing by the total number of possible sequences.

```
// Probability of being at state s after n random steps.
// ways[n][s] = number of paths of length n ending at s.
// Probability = ways[n][s] / total_ways.
vector<vector<ll>> ways(n+1, vector<ll>(S, 0));
ways[0][start] = 1;
for(int i=0;i<n;i++)
    for(int s=0;s<S;s++)
        for(int t : successors(s))
            ways[i+1][t] = (ways[i+1][t] + ways[i][s]) % MOD;
ll total = modpow(deg, n);            // or some appropriate total
ll prob = ways[n][target] * inv(total) % MOD;
```

This is a common bridge between counting DP and probability DP.

13.8 Variance, second moments

To compute the *variance* of a quantity, we also need the second moment: $\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$. The second moment requires maintaining $E[X^2]$ alongside $E[X]$, with a more involved recurrence.

For a sum $S = \sum_i X_i$ of independent variables, $\mathrm{Var}(S) = \sum_i \mathrm{Var}(X_i)$, which is far easier than the general case.

13.9 Problem set

P1. Codeforces 1265E, Beautiful Mirrors. n mirrors each with success probability p_i. Each day you visit the mirrors in order; on success you proceed; on failure you restart from mirror 1. Expected number of days to reach the end. *Hint:* E_i = (expected days from mirror i to mirror $n+1$); $E_i = (1 + p_i E_{i+1} + (1 - p_i)(1 + E_1))$.

P2. Codeforces 1349D, Mandarines and Coins. Probability DP with state being "number of remaining coins".

P3. Codeforces 804D, Expected Number of Components. Linearity of expectation on connected components.

P4. Codeforces 1486D, Max Median. Hybrid with binary search + counting DP.

P5. Codeforces 1426D, De Bruijn Sequence. Hybrid with string DP.

P6. Codeforces 1450C2, Colliding Encodings. Probability of winning a tournament.

P7. AtCoder ABC 134 E, Sequence Decomposing. Greedy + DP hybrid.

P8. AtCoder ABC 240 F, Sum of Sum of Digits. Hybrid with digit DP.

P9. Codeforces 1452C, Slime Game. Two-player zero-sum game with mixed strategies; reduce via minimax DP.

13.10 Common pitfalls

- **Forgetting to divide by the total.** A "probability" is a count divided by a total. Always track the total number of equal-probability outcomes and divide by it.
- **Modular inverse of zero.** If the total can be 0 (degenerate cases), skip those states explicitly.
- **Wrong base case for expected value.** $E[\text{terminal}] = 0$, not "the value at the terminal".
- **Cycles in random-walk DP.** If the graph has cycles, you cannot simply recurse; use Gaussian elimination or bounded-step DP.
- **Precision.** For floating-point expected value, do *not* work modulo p — use `long double` and verify with stress tests. Most modern contests ask for the answer mod p instead.

Chapter 14

DP Optimisations

14.1 Why a dedicated optimisation chapter

Many DP solutions have an obvious $O(n^2)$ or $O(n^3)$ recurrence that TLEs on the contest's largest constraints. In most cases, the *recurrence structure* already satisfies additional mathematical properties (convexity, monotonicity, Monge / quadrangle inequality) that admit a generic optimisation to $O(n \log n)$ or $O(n^2)$. This chapter catalogues the standard optimisations:

1. **Prefix-sum / range-query optimisation.** Convert inner loop sums into $O(1)$ Fenwick / segment-tree lookups. Section 14.2.
2. **Monotonic queue / sliding window optimisation.** For DPs whose inner transition is a sliding-window maximum. Section 14.3.
3. **Divide-and-conquer optimisation.** For DPs whose optimal cut point $opt[i][j]$ is monotone in j. Section 14.4.
4. **Knuth optimisation.** For DPs satisfying the quadrangle inequality. Section 14.5.
5. **Convex Hull Trick (CHT).** For DPs of the form $dp[i] = \min_j\{m_j \cdot x_i + b_j\}$. Section 14.6.
6. **Li Chao segment tree.** A generalisation of CHT to arbitrary x_i order. Section 14.7.
7. **Alien's trick (Lagrangian reduction).** Add a Lagrange multiplier to enforce a cardinality constraint. Section 14.8.

14.2 Prefix-sum optimisation

14.2.1 Pattern

Many DPs have a transition of the form

$$dp[i] = \min_{j<i}\big(dp[j] + S(j+1, i)\big),$$

where $S(l, r)$ is the sum of $a[l..r]$. Naively each i requires $O(i)$ transitions, giving $O(n^2)$. With a prefix sum

$$P[i] = \sum_{k=1}^{i} a[k], \quad S(l, r) = P[r] - P[l-1],$$

the sum is $O(1)$ but the inner loop is still $O(i)$.

14.2.2 When prefix sums are enough

Prefix sums reduce the transition from $O(n)$ (summing a) to $O(1)$, but they do not reduce the *number of candidate j's*. The full $O(n^2)$ remains. Prefix sums are usually a necessary prerequisite for the heavier optimisations below (CHT, D&C, etc.).

```
vector<ll> P(n+1, 0);
for(int i=1;i<=n;i++) P[i] = P[i-1] + a[i];
auto S = [&](int l, int r){ return P[r] - P[l-1]; };
for(int i=1;i<=n;i++){
    dp[i] = INF;
    for(int j=0;j<i;j++)
        dp[i] = min(dp[i], dp[j] + S(j+1, i));
}
```

14.3 Monotonic queue / sliding-window optimisation

14.3.1 Pattern

For DPs of the form

$$dp[i] = \min_{j \in [i-k, i-1]} (dp[j] + a[j]),$$

i.e. the inner range is a fixed-width sliding window, a *monotonic queue* keeps the minimum in $O(1)$ amortised.

```
deque<int> dq;       // indices, dp values monotone increasing from front to back
for(int i=0;i<=n;i++){
    // Push dp[i-1] (or the appropriate state) into the queue with the
    // sliding-window constraint.
    while(!dq.empty() && dp[dq.back()] >= dp[i-1]) dq.pop_back();
    dq.push_back(i-1);
    while(!dq.empty() && dq.front() < i - k) dq.pop_front();
    dp[i] = (dq.empty() ? INF : dp[dq.front()] + a[i-1]);
}
```

14.3.2 Monotonic queue knapsack (bounded knapsack, $O(nW)$)

The bounded knapsack problem (each item i usable up to m_i times, Section 3.4) can be solved in true $O(nW)$ by grouping the DP entries by their index modulo w_i and applying a sliding-window minimum on each group. The idea:

- For each item i and each residue $r \in [0, w_i)$, the states $dp[r], dp[r + w_i], dp[r + 2w_i], \ldots$ form a 1-D sequence, and the update is a bounded sliding-window maximum (with the offset $+v_i$).
- Apply the monotonic queue on each residue class separately.

```
// Bounded knapsack in O(n * W) using monotonic queue.
vector<ll> dp(W+1, 0);
for(int i=0;i<n;i++){
    ll w = wt[i], v = val[i], m = cnt[i];
    vector<ll> ndp = dp;
    for(int r = 0; r < w; r++){
        deque<int> dq;
        for(ll c = r, k = 0; c <= W; c += w, k++){
            // candidate: dp[c] (which still represents the previous item's row)
            while(!dq.empty() && dp[dq.back()] - (dq.back() - r) / w * v
                                 <= dp[c] - (c - r) / w * v) dq.pop_back();
            dq.push_back(c);
            while(!dq.front() < c - m * w) dq.pop_front();
            ndp[c] = max(ndp[c], dp[dq.front()] + (c - dq.front()) / w * v);
        }
    }
    dp = ndp;
}
```

This is the standard solution to Leetcode's bounded knapsack and the SPOJ MKBUDGET problem.

14.4 Divide-and-conquer optimisation

14.4.1 Pattern

Consider a DP of the form

$$dp[i][j] = \min_{k<j}(dp[i-1][k] + C(k+1, j)),$$

where C is some cost function. Naive: $O(n^2 \cdot m)$ for m rows. The D&C optimisation applies when the optimal cut $opt[i][j]$ is *monotone* in j: $opt[i][j] \leq opt[i][j+1]$. Under this condition, we can compute $dp[i][\cdot]$ for a fixed i in $O(n \log n)$ rather than $O(n^2)$.

14.4.2 Algorithm

1. For fixed i, recursively compute $dp[i][j]$ over the range $j \in [l, r]$.
2. Pick the midpoint $m = (l+r)/2$. Compute $opt[i][m]$ by trying all $k \in [opt[i][l-1], opt[i][r+1]]$ (this is the monotone window).
3. Recurse into $[l, m-1]$ (using the upper bound $opt[i][m]$) and $[m+1, r]$ (using the lower bound $opt[i][m]$).

Because the search ranges shrink geometrically, the total work is $O(n \log n)$ per row.

```
void compute_row(int i, int l, int r, int optL, int optR){
    if(l > r) return;
    int m = (l + r) / 2;
    int best_k = -1;
    ll best = INF;
    for(int k = optL; k <= min(m-1, optR); k++){
        ll cand = dp_prev[k] + cost(k+1, m);
        if(cand < best){ best = cand; best_k = k; }
    }
    dp_cur[m] = best;
    opt[m] = best_k;
    compute_row(i, l, m-1, optL, best_k);
    compute_row(i, m+1, r, best_k, optR);
}
```

Tip 14.1: When is D&C opt valid?

D&C optimisation is valid iff the cost function C satisfies the *quadrangle inequality* (a.k.a. the Monge property) and is *monotone in the upper bound*. A practical test: write a brute $O(n^2)$ DP on small inputs, dump $opt[i][j]$ for every (i, j), and verify that $opt[i][j]$ is non-decreasing in j. If yes, D&C applies.

14.5 Knuth optimisation

14.5.1 Pattern

For interval DPs of the form

$$dp[l][r] = \min_{k \in [l, r-1]}(dp[l][k] + dp[k+1][r] + C(l, r, k)),$$

where the cost C satisfies the *quadrangle inequality* and *monotonicity*, the optimal cut point $K[l][r]$ satisfies

$$K[l][r-1] \le K[l][r] \le K[l+1][r].$$

This shrinks the search range to a constant-size window on average, reducing the total work from $O(n^3)$ to $O(n^2)$.

14.5.2 Quadrangle inequality

A cost function C satisfies the *quadrangle inequality* (QI) if for all $a \le b \le c \le d$,

$$C(a,c) + C(b,d) \le C(a,d) + C(b,c).$$

And C is *monotone* if $C(b,c) \le C(a,d)$ for $a \le b \le c \le d$. Together these imply the Knuth bound.

14.5.3 Examples satisfying QI

- **Matrix chain multiplication:** $C(l,r,k) = p_{l-1}p_kp_r$. Satisfies QI.
- **Optimal BST:** $C(l,r) = \sum_{i=l}^{r} f_i + \sum_{i=l-1}^{r} d_i$ (constant in k). Satisfies QI.
- **Merging stones (any-two-adjacent rule):** $C(l,r) = \text{sum}[l..r]$ (constant in k). Satisfies QI.

14.5.4 C++17 template

```cpp
// Knuth-optimised interval DP. K[l][r] stores the best cut.
for(int len=1; len<=n; len++)
    for(int l=1; l+len-1<=n; l++){
        int r = l + len - 1;
        if(len == 1){ dp[l][r] = 0; K[l][r] = l; continue; }
        int lo = K[l][r-1];
        int hi = (r+1<=n ? K[l+1][r] : r-1);
        dp[l][r] = INF;
        for(int k=lo; k<=hi; k++){
            ll cand = dp[l][k] + dp[k+1][r] + cost(l, r, k);
            if(cand < dp[l][r]){ dp[l][r] = cand; K[l][r] = k; }
        }
    }
```

The total work is $\sum_{len} \sum_{l} (K[l+1][r] - K[l][r-1] + 1) \le \sum_{len} O(n) = O(n^2)$.

14.6 Convex Hull Trick

14.6.1 Pattern

For DPs of the form

$$dp[i] = \min_{j<i} (m_j \cdot x_i + b_j),$$

where the value being minimised is a linear function of x_i with slope m_j and intercept b_j that depend on j. Naive: $O(n^2)$. The Convex Hull Trick reduces to $O(n \log n)$ (general case) or $O(n)$ (if x_i is monotone and m_j is monotone).

14.6.2 Lower envelope of lines

We maintain the *lower envelope* of the lines $y = m_j x + b_j$ as a deque. Each line is pushed in m-order; we remove the last line whenever the new line makes it redundant.

14.6.3 Monotone case (x and m both monotone)

If both the queries x_i and the slopes m_j are monotone (say both increasing), the deque is queried from the front and updated from the back, both $O(1)$ amortised. Total $O(n)$.

```
struct Line { ll m, b; };
// We want lower envelope; slopes added in decreasing order.
ll eval(Line L, ll x){ return L.m * x + L.b; }
ll intersect(Line a, Line b){              // x-coordinate of intersection
    return (b.b - a.b) / (a.m - b.m);      // careful with floor division
}
deque<Line> hull;
auto add = [&](Line nl){
    while(hull.size() >= 2){
        Line a = hull[hull.size()-2], b = hull.back();
        if(eval(b, intersect(a, b)) >= eval(nl, intersect(a, b)))
            hull.pop_back();
        else break;
    }
    hull.push_back(nl);
};
auto query = [&](ll x){
    while(hull.size() >= 2 && eval(hull[0], x) >= eval(hull[1], x))
        hull.pop_front();
    return eval(hull[0], x);
};
```

14.6.4 Non-monotone case (Li Chao)

When the queries are not monotone in x, the CHT can still work but needs a binary search on the hull's x-intersection points. The cleaner alternative is the **Li Chao segment tree** of Section 14.7.

14.6.5 Canonical example

The DP $dp[i] = \min_{j<i}\{dp[j] + (a[i] - a[j])^2\}$ expands to $dp[j] + a[j]^2 - 2a[i]a[j] + a[i]^2$. Treating $x = a[i]$ and writing the inner term as $(-2a[j]) \cdot x + (dp[j] + a[j]^2)$, we have a line in x with slope $m_j = -2a[j]$ and intercept $b_j = dp[j] + a[j]^2$. So the CHT applies directly.

14.7 Li Chao segment tree

14.7.1 Pattern

Same as CHT but queries are arbitrary (not monotone in x). Li Chao segment tree supports inserting a line and querying the minimum at any x in $O(\log C)$ where C is the range of x.

14.7.2 Algorithm

The segment tree partitions the x-axis into $O(\log C)$ disjoint segments per query / update. Each node stores the line that is the "lower" of the two lines at the segment's midpoint; the other line is recursively pushed down into the half-segment where it might still be the lower envelope.

```
struct LiChao {
    struct Line { ll m, b; ll eval(ll x){ return m*x+b; } };
    int n; vector<Line> tree;
    LiChao(int n_, ll lo, ll hi): n(n_), tree(4*n_, {0, LLONG_MAX/4}) {}
    void add_line(int v, int tl, int tr, Line nl){
        int tm = (tl + tr) / 2;
        bool lef = nl.eval(tl) < tree[v].eval(tl);
```

```
        bool mid = nl.eval(tm) < tree[v].eval(tm);
        if(mid) swap(tree[v], nl);
        if(tr - tl == 1) return;
        if(lef != mid) add_line(2*v, tl, tm, nl);
        else           add_line(2*v+1, tm, tr, nl);
    }
    ll query(int v, int tl, int tr, ll x){
        int tm = (tl + tr) / 2;
        ll ans = tree[v].eval(x);
        if(tr - tl == 1) return ans;
        if(x < tm) return min(ans, query(2*v, tl, tm, x));
        else       return min(ans, query(2*v+1, tm, tr, x));
    }
};
```

The tree is built on the range $[lo, hi]$ of possible x values; the size is 4 times the coordinate range after compression.

14.8 Alien's trick (Lagrangian relaxation)

14.8.1 Pattern

Consider a problem "find the minimum cost to achieve objective $\geq T$". If we know how to solve the easier problem "find the minimum cost without constraint on T, where each step incurs an additional penalty λ", we can binary-search λ such that the unconstrained solution has exactly T steps. The Lagrangian dual guarantees this works when the value function is convex in T.

14.8.2 Canonical example: split array into k subarrays with minimum total cost

Given $a[1..n]$ and k, partition into exactly k contiguous subarrays minimising the sum of per-subarray costs. Direct DP: $O(n^2 k)$. Alien's trick reduces to a 1-D DP $O(n \log C)$ per λ value, and binary-searches λ in $O(\log C)$ iterations: $O(n \log^2 C)$ total.

14.8.3 Algorithm

1. For each candidate λ (the per-subarray penalty), run a 1-D DP $dp[i]$ = minimum cost to partition $a[1..i]$, where each subarray incurs an additional $-\lambda$ (or $+\lambda$). Also track $cnt[i]$ = number of subarrays used.
2. Binary-search λ so that $cnt[n] = k$. The final answer is $dp[n] + k \cdot \lambda$ (since the penalty should be added back).

```
// Partition a[1..n] into exactly k subarrays; cost of [l..r] = cost(l,r).
// Alien's trick: solve without k, charge lambda per subarray, binary search lambda.
pair<ll,ll> dp_with_penalty(ll lambda){
    // dp[i] = min cost to partition a[1..i] with per-subarray penalty lambda.
    // cnt[i] = number of subarrays used.
    vector<ll> dp(n+1, INF); vector<int> cnt(n+1, 0);
    dp[0] = 0;
    for(int i=1;i<=n;i++){
        for(int j=0;j<i;j++){
            ll cand = dp[j] + cost(j+1, i) + lambda;
            if(cand < dp[i]){ dp[i] = cand; cnt[i] = cnt[j] + 1; }
        }
    }
    return {dp[n], cnt[n]};
}
// Binary search lambda in [-C, C]; check (dp, cnt) for each.
```

Warning 14.1: Alien's trick assumptions

Alien's trick requires the value function $f(k)$ = "min cost with exactly k subarrays" to be *convex* in k. If it is not (e.g. non-monotone costs), the binary-search of λ can give wrong results. Always verify convexity by brute-forcing small inputs.

14.9 Other optimisations (sketches)

14.9.1 Slope trick

For DPs whose "derivative" $dp[i+1]-dp[i]$ is monotone, the sequence of slopes can be maintained with a priority queue (the "slope trick"). Typical applications: "k-server problem", "slope trick on path graphs".

14.9.2 SQRT decomposition on states

When the state space is too large but admits a natural partition into B groups of size n/B, choose $B = \sqrt{n}$ and amortise.

14.9.3 Heavy-light decomposition on tree DP

When a tree DP needs to handle updates to a single vertex and queries on root-to-leaf paths, HLD with segment tree gives $O(\log^2 n)$ per operation.

14.10 Problem set

P1. Codeforces 321E, Ciel and G-Strings. Alien's trick.

P2. Codeforces 868E, Polly's Cheat. Alien's trick on tree.

P3. SPOJ NKSUBSTR / ATCODER ABC 234 F. Prefix-sum-optimised DP.

P4. Codeforces 1301F, Air Conditioners. D&C optimisation.

P5. Codeforces 528D, Tree and Queries. CHT on monotone slope.

P6. Codeforces 1343F, Restore the Permutation. CHT with non-monotone x via Li Chao tree.

P7. Codeforces 1342E, Placing Rooks. Alien's trick on a combinatorial problem (rook polynomial).

P8. Codeforces 1175G, Red and Blue. CHT on the lower envelope of a quadratic function.

P9. AtCoder ABC 217 H, Greedy Tak. Alien's trick.

P10. Codeforces 1178G, The Orange Tree. Li Chao tree for online queries.

14.11 Common pitfalls

- **Checking the QI condition.** Knuth and D&C optimisations *require* the cost function to satisfy the quadrangle inequality. Verify it on small inputs before applying.
- **Integer division in CHT intersections.** Use floating point for the intersection test, or carefully use `ll` with floor division.
- **Alien's trick without convexity.** Without convexity, the binary search of λ returns an answer that does not correspond to k subarrays.
- **Off-by-one in Li Chao recursion.** The base case `tr - tl == 1` returns the line at the leaf. The recursion is on the *half-interval*, not on the midpoint.
- **Choosing the right optimisation.** Many problems admit *several* optimisations. The hierarchy (cheap to expensive) is: prefix sums $\to$ monotonic queue $\to$ Knuth / D&C $\to$ CHT $\to$ Li Chao $\to$ alien's trick. Try the cheapest first.

Chapter 15

Conclusion, Master Checklist, and Pitfalls

15.1 What you have learned

This book has covered the eleven standard DP families that account for the overwhelming majority of contest DP problems:

1. **The Knapsack family** (Chapter 3): 0/1, unbounded, bounded, subset sum, partition, count of subsets. The state is (i, c) where i is an item index and c a capacity-like parameter.
2. **The LIS / LCS family** (Chapter 4): state is one or two indices into a sequence; transitions add one element to a growing subsequence. The $O(n \log n)$ patience algorithm is the optimisation everyone should memorise.
3. **Interval DP** (Chapter 5): state is $[l, r]$. Used when the problem asks "best order to combine contiguous pieces". $O(n^3)$ naive, $O(n^2)$ with Knuth.
4. **Tree DP** (Chapter 6): state is a subtree rooted at u; transitions merge children. Rerooting computes the answer for *every* root in $O(n)$.
5. **DAG / path-counting DP** (Chapter 7): state is a vertex of a DAG, transitions follow edges. SOS DP is a specialisation.
6. **Bitmask DP** (Chapter 8): state is a subset encoded as a bitmask. Used when $n \leq 20$ and the problem is defined over subsets.
7. **Digit DP** (Chapter 9): state is a digit position plus a tight flag. Used for ranges up to 10^{18}.
8. **Game DP / Sprague–Grundy** (Chapter 10): state is a game position; Grundy value is the mex of successors' Grundy values.
9. **Matrix exponentiation** (Chapter 11): for linear recurrences of bounded degree, $O(k^3 \log n)$ instead of $O(n)$ DP.
10. **String / automaton DP** (Chapter 12): state is an index into a string *and* a vertex of an automaton (KMP, AC, SAM).
11. **Probability / expected value DP** (Chapter 13): state values are probabilities or expectations; merges are expectation-preserving, with modular inverse replacing division.

On top of these families, the optimisations of Chapter 14—prefix sums, monotonic queue, D&C, Knuth, CHT, Li Chao, alien's trick—give the standard bag of speed-ups that take an $O(n^2)$ or $O(n^3)$ DP down to $O(n \log n)$ or $O(n^2)$.

15.2 Master recognition checklist

When you read a fresh contest problem, walk through this checklist in order. If you can answer the questions confidently, you have identified the DP family.

DP recognition master checklist

1. Constraints. Read the constraint on every variable. Look up the largest n in the table of Section 2.2. What complexity does it suggest?
2. Keywords. Read the problem statement and identify at least one keyword from the lexicon of Section 2.3. What DP family does it suggest?
3. Shape. Identify the shape of the input: 1-D array / pair of indices / interval / subset / tree / DAG / digit string / automaton / game position.
4. Four ingredients. Write down (i) state, (ii) transition, (iii) base case, (iv) answer. If any one is fuzzy, you do not yet have a DP — go back to step 1.
5. Family mapping. Use the shape to pick a chapter of this book:

Shape	Chapter
1-D array with capacity parameter	Knapsack (Ch. 3)
1-D array, no capacity	LIS/LCS or linear DP (Ch. 4)
Pair of indices	LCS / edit distance (Ch. 4)
Interval $[l, r]$	Interval DP (Ch. 5)
Subset	Bitmask DP (Ch. 8)
Tree	Tree DP (Ch. 6)
DAG / grid paths	DAG DP (Ch. 7)
Integer range $[L, R]$ up to 10^{18}	Digit DP (Ch. 9)
String + automaton	String DP (Ch. 12)
Game position	Game DP (Ch. 10)
Linear recurrence	Matrix exponentiation (Ch. 11)

6. Complexity check. Multiply state count $\times$ transition count. Is it $\leq 10^8$? If not, do you need an optimisation from Chapter 14?
7. Sanity. Greedy alternative? Closed form? NP-hardness check?

15.3 Common pitfalls (cross-cutting)

- **Wrong loop order.** The direction of the inner loop is the difference between 0/1 and unbounded knapsack, between ordered and unordered compositions, and between correctly populating and incorrectly overwriting the DP table. Always sanity-check the loop direction with a 3-state example.
- **Stack overflow.** For $n \geq 10^5$ on a deep recursion, either `ulimit -s unlimited` or rewrite the DFS iteratively.
- **Modulo arithmetic.** Use `long long` for all intermediate products; take `%MOD` after every addition and every multiplication. Special care for negative mod results (always add `MOD` before taking the modulus).
- **Index off-by-one.** The DP table usually has an extra row / column for the "empty prefix". Decide once whether your code is 0-indexed or 1-indexed and stick to it.
- **Memory limits.** A `long long` table of size 5000×5000 is 200 MB. Many problems set the memory limit at 256 MB or 512 MB — be careful. Use rolling arrays or Hirschberg's algorithm to reduce memory.

- **Recurrence vs greedy.** Before writing a DP, construct two or three small adversarial tests for the obvious greedy. If any greedy fails on them, DP is the right tool.
- **Wrong answer on base cases.** A common bug: forgetting that $dp[0]$ = base case, not $dp[1]$. Always print the DP table on a 5-state example to inspect.
- **Wrong integer type.** Use `long long` whenever the answer can exceed $2 \cdot 10^9$. Use `unsigned long long` only when necessary (rare).
- **Wrong bitset size.** For subset-sum bitset DP, ensure the bitset has size $\geq \max a_i + \sum a_i$ if shifts can accumulate beyond $\max a_i$.
- **Wrong transition direction in interval DP.** Always iterate by length-first; never by starting position.
- **Forgetting the tight flag in digit DP.** The tight flag is part of the state; ignoring it makes the memo table return wrong values.
- **Confusing failure link with suffix link.** KMP's failure function and SAM's suffix link are different objects.

15.4 A workflow for contest DP problems

When you sit down with a contest problem you suspect is DP, follow this workflow:

1. *Read the constraints.* Write them down next to the problem. Compute the state space × transition cost. Is the product $\leq 10^8$?
2. *Identify the keyword(s) and shape.* Use the lexicon and the shape taxonomy from Chapter 2.
3. *Pick a chapter* using the family mapping table above.
4. *Write down the four ingredients* (state, transition, base, answer) on the back of the problem sheet.
5. *Sanity check against greedy and closed form.*
6. *Code the top-down DP first*; submit; iterate on edge cases.
7. *If TLE/MLE*, apply the cheapest applicable optimisation from Chapter 14: prefix sums first, then monotonic queue, then D&C / Knuth, then CHT, then Li Chao, finally alien's trick.
8. *Keep the editorial and your code* — these are reusable assets.

15.5 Where to go next

This book covers the standard DP curriculum for competitive programming. Beyond it, the following topics are useful for advanced contests:

- **Data-structure-bosted DP.** Segment tree, Fenwick, and `set` augmented with DP transitions.
- **Heavy-light decomposition** on top of tree DP.
- **Link-cut trees** for dynamic tree DP.
- **Subset convolution** and **ranked SOS DP** for advanced subset DP.
- **Cayley–Hamilton** reduction for fast computation of linear recurrences with large k.
- **Misère quotient theory** for misère impartial games.

- **Conway's surreal numbers** for partisan games.

Each of these is a topic in its own right; the references below are starting points.

15.6 References

The following resources complement this book:

- **Competitive Programmer's Handbook** (Antti Laaksonen): a free online textbook covering the standard contest DP families with worked examples.
- **AtCoder DP Contest** (`https://atcoder.jp/contests/dp`): a curated set of ~ 30 problems covering every family discussed in this book.
- **CSES Problem Set** (`https://cses.fi/problemset/`): especially the "Dynamic Programming" section.
- **Codeforces DP blog series** by various authors; in particular the "Dynamic programming: from novice to advanced" tutorial.
- **USACO Guide** (`https://usaco.guide/`): a free graded curriculum including the DP chapters.
- **cp-algorithms.com**: reference for all standard algorithms, including DP optimisations.
- **Winning Ways for your Mathematical Plays** (Berlekamp, Conway, Guy): the canonical reference for combinatorial game theory beyond Sprague–Grundy.

15.7 Final words

Dynamic programming rewards pattern recognition. The single most important habit you can build is the one taught in Chapter 2: read the constraints, read the keywords, identify the shape, write down the four ingredients, and only then begin coding. If you internalise that habit, the rest of the contest DP curriculum becomes an exercise in matching problems to chapters of this book. Good hunting.